US006825851B1

(12) **United States Patent**
Leather

(10) **Patent No.: US 6,825,851 B1**
(45) **Date of Patent: Nov. 30, 2004**

(54) **METHOD AND APPARATUS FOR ENVIRONMENT-MAPPED BUMP-MAPPING IN A GRAPHICS SYSTEM**

(75) Inventor: **Mark M. Leather**, Saratoga, CA (US)

(73) Assignee: **Nintendo Co., Ltd.**, Kyoto (JP)

(*) Notice: Subject to any disclaimer, the term of this patent is extended or adjusted under 35 U.S.C. 154(b) by 369 days.

(21) Appl. No.: **09/722,381**

(22) Filed: **Nov. 28, 2000**

Related U.S. Application Data

(60) Provisional application No. 60/226,893, filed on Aug. 23, 2000.

(51) **Int. Cl.**[7] **G09G 5/00**; G06T 15/60

(52) **U.S. Cl.** **345/584**; 345/581; 345/426; 345/848

(58) **Field of Search** 345/426, 581–582, 345/584, 501, 506, 427, 522, 848

(56) **References Cited**

U.S. PATENT DOCUMENTS

4,388,620 A 6/1983 Sherman

(List continued on next page.)

FOREIGN PATENT DOCUMENTS

CA 2070934 12/1993
EP 0 637 813 A2 2/1995

(List continued on next page.)

OTHER PUBLICATIONS

Heidrich et al., "Illuminating Micro Geometry Based on Precomputed Visiblity," Computer Graphics Proceedings, Annual Conference Series, 2000, pp. 455–464.

(List continued on next page.)

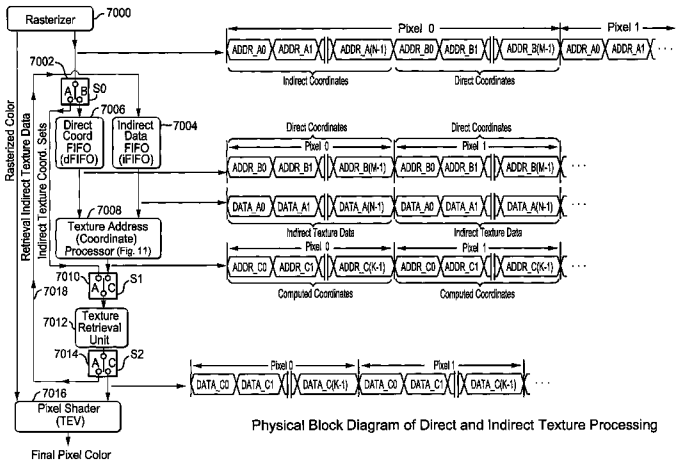
Primary Examiner—Matthew C. Bella
Assistant Examiner—Anthony Blackman

(74) *Attorney, Agent, or Firm*—Nixon & Vanderhye PC

(57) **ABSTRACT**

A graphics system including a custom graphics and audio processor produces exciting 2D and 3D graphics and surround sound. The system includes a graphics and audio processor including a 3D graphics pipeline and an audio digital signal processor. Realistic looking surfaces on rendered images are generated by EMBM using an indirect-texture lookup to a "bump map" followed by an environment or light mapping. Apparatus and example methods for environment-mapped style of bump-mapping (EMBM) are provided that use a pre-completed bump-map texture accessed as an indirect texture along with pre-computed object surface normals (i.e., the Normal, Tangent and Binormal vectors) from each vertex of rendered polygons to effectively generate a new perturbed Normal vector per vertex. The perturbed new Normal vectors are then used to look up texels in an environment map which, for example, may be a spherical reflection map of the environment surrounding a rendered object or a texture map containing lighting or other data for creating more complex surface lighting effects. A specialized bump map texture data/coordinate processing "bump unit" is provided in the graphics pipeline for performing predetermined matrix multiplication operations on retrieved lookup data from the indirect-texture bump map. The bump unit utilizes a programmable 3×2 element multiplication matrix and scaling circuits that permit rotation of bump-mapped objects about multiple orthogonal axes. The bump unit is also provided with an output storage buffer for reusing computed coordinate data values in subsequent processing cycles.

6 Claims, 37 Drawing Sheets



Physical Block Diagram of Direct and Indirect Texture Processing

U.S. PATENT DOCUMENTS

4,425,559 A 1/1984 Sherman
4,570,233 A 2/1986 Yan et al.
4,615,013 A 9/1986 Yan et al.
4,658,247 A 4/1987 Gharachorloo
4,725,831 A 2/1988 Coleman
4,829,295 A 5/1989 Hiroyuki
4,862,392 A 8/1989 Steiner
4,866,637 A 9/1989 Gonzalez-Lopez et al.
4,901,064 A 2/1990 Deering
4,914,729 A 4/1990 Omori et al.
4,945,500 A 7/1990 Deering
5,136,664 A 8/1992 Bersack et al.
5,170,468 A 12/1992 Shah et al.
5,392,385 A 2/1995 Evangelisti et al.
5,392,393 A 2/1995 Deering
5,421,028 A 5/1995 Swanson
5,422,997 A 6/1995 Nagashima
5,457,775 A 10/1995 Johnson, Jr. et al.
5,504,917 A 4/1996 Austin
5,544,292 A 8/1996 Winser
5,561,746 A 10/1996 Murata et al.
5,561,756 A * 10/1996 Miller et al. 345/848
5,563,989 A 10/1996 Billyard
5,594,854 A 1/1997 Baldwin et al.
5,608,424 A 3/1997 Takahashi et al.
5,687,357 A 11/1997 Priem
5,696,892 A * 12/1997 Redmann et al. 345/582
5,701,444 A 12/1997 Baldwin
5,721,947 A 2/1998 Priem et al.
5,727,192 A 3/1998 Baldwin
5,751,292 A 5/1998 Emmot
5,758,182 A 5/1998 Rosenthal et al.
5,764,243 A 6/1998 Baldwin
5,768,626 A 6/1998 Munson et al.
5,768,629 A 6/1998 Wise et al.
5,774,133 A 6/1998 Neave et al.
5,777,629 A 7/1998 Baldwin
5,798,770 A 8/1998 Baldwin
5,801,706 A 9/1998 Fujita et al.
5,801,716 A 9/1998 Silverbrook
5,805,868 A 9/1998 Murphy
5,808,619 A 9/1998 Choi et al.
5,809,219 A 9/1998 Pearce et al.
5,815,166 A 9/1998 Baldwin
5,821,949 A 10/1998 Deering
5,870,102 A 2/1999 Tarolli et al.
5,874,969 A 2/1999 Storm et al.
5,880,736 A 3/1999 Peercy et al.
5,917,496 A 6/1999 Fujita et al.
5,920,326 A 7/1999 Rentschler et al.
5,923,334 A 7/1999 Luken
5,940,086 A 8/1999 Rentschler et al.
5,949,421 A 9/1999 Ogletree et al.
5,949,424 A 9/1999 Cabral et al.
5,949,440 A 9/1999 Krech, Jr. et al.
5,956,043 A 9/1999 Jensen
5,969,726 A 10/1999 Rentschler et al.
5,999,196 A 12/1999 Storm et al.
6,002,409 A 12/1999 Harkin
6,023,738 A 2/2000 Priem et al.
6,025,853 A 2/2000 Baldwin
6,028,611 A 2/2000 Anderson et al.
6,037,949 A 3/2000 DeRose et al.
6,049,337 A 4/2000 Van Overveld
6,052,127 A 4/2000 Vaswani et al.
6,057,851 A 5/2000 Luken et al.
6,057,852 A 5/2000 Krech, Jr.
6,078,333 A 6/2000 Wittig et al.
6,078,334 A 6/2000 Hanaoka et al.
6,092,124 A 7/2000 Priem et al.
6,104,415 A 8/2000 Gossett
6,163,319 A * 12/2000 Peercy et al. 345/426
6,173,367 B1 1/2001 Aleksic et al.
6,181,352 B1 1/2001 Kirk et al.
6,191,794 B1 2/2001 Priem et al.
6,198,488 B1 3/2001 Lindholm et al.
6,226,012 B1 5/2001 Priem et al.
6,229,553 B1 5/2001 Duluk et al.
6,259,460 B1 7/2001 Gossett et al.
6,337,689 B1 1/2002 Hochmuth et al.
6,353,438 B1 3/2002 Van Hook et al.
6,384,824 B1 * 5/2002 Morgan et al. 345/426
6,426,747 B1 7/2002 Hoppe et al.
6,459,429 B1 10/2002 Derring
6,466,223 B1 * 10/2002 Dorbie et al. 345/582
6,501,479 B1 12/2002 Root et al.
6,593,923 B1 7/2003 Donovan et al.
6,597,363 B1 7/2003 Duluk et al.
6,664,958 B1 12/2003 Leather et al.

FOREIGN PATENT DOCUMENTS

EP 1 074 945 2/2001
EP 1 075 146 2/2001
EP 1 081 649 3/2001
EP 1 189 172 A9 3/2002
JP 11053580 2/1999
JP 11076614 3/1999
JP 11161819 6/1999
JP 11203500 7/1999
JP 11226257 8/1999
JP 11259671 9/1999
JP 11259678 9/1999
JP 2000-66985 3/2000
JP 2000-92390 3/2000
JP 2000-132704 5/2000
JP 2000-132706 5/2000
JP 2000-149053 5/2000
JP 2000-156875 6/2000
JP 2000-182077 6/2000
JP 2000-207582 7/2000
JP 2000-215325 8/2000
WO WO 94/10641 5/1994

OTHER PUBLICATIONS

McCool et al., "Texture Shaders," Proceedings 1999 Eurographics Workshop, Los Angeles, CA, Aug. 8–9, 1999, pp. 117–126; 144.

Photograph of Sony PlayStation II System.

Photograph of Sega Dreamcast System.

Photograph of Nintendo 64 System.

Whitepaper: 3D Graphics Demystified, Nov. 11, 1999, www.nvidia.com.

Whitepaper: "Z Buffering, Interpolation and More W–Buffering", Doug Rogers, Jan. 31, 2000, www.nvidia.com.

Whitepaper: Using GL_NV_vertex_array and GL_NV_fence, posted Aug. 1, 2000, www.nvidia.com.

Whitepaper: Anisotropic Texture Filtering in OpenGL, posted Jul. 17, 2000, www.nvidia.com.

Whitepaper: Mapping Texels to Pixels in D3D, posted Apr. 5, 2000, www.nvidia.com.

Whitepaper: Guard Band Clipping, posted Jan. 31, 2000, www.nvidia.com.

Whitepaper: Cube Environment Mapping, posted Jan. 14, 2000, www.nvidia.com.

Whitepaper: Color Key in D3D, posted Jan. 11, 2000, www.nvidia.com.

Whitepaper: Vertex Blending Under DX7 for the GeForce 256, Jan. 5, 2000, www.nvidia.com.
Whitepaper: Optimizing Direct3D for the GeForce 256, Jan. 3, 2000, www.nvidia.com.
Whitepaper: Dot Product Texture Blending, Dec. 3, 1999, www.nvidia.com.
Whitepaper: Technical Brief: AGP 4X with Fast Writes, Nov. 10, 1999, www.nvidia.com.
Technical Brief: Transform and Lighting, Nov. 10, 1999, www.nvidia.com.
Technical Brief: What's New With Microsoft DirectX7, posted Nov. 10, 1999, www.nvidia.com.
Mitchell et al., "Multitexturing in DirectX6", Game Developer, Sep. 1998, www.gdmag.com.
VisionTek, "GeForce2 GS Graphics Processing Unit", ©2000 www.visiontek.com.
Jim Bushnell et al. "Advanced Multitexture Effects With Direct3D and OpenGL", Pyramid Peak Design & ATI Research, Inc., GameDevelopers Conference, ©1999.
Sony PlayStation II Instruction Manual, Sony Computer Entertainment Inc., ©2000.
Stand and Be Judged, Next Generation, May 2000.
PlayStation II: Hardware Heaven or Hell?, Next Generation, Jan. 2000.
Chris Charla, "Play Station II: The Latest News", Next Generation, Sep. 1999.
"First PlayStation II Gameplay Screens Revealed!", Next Generation, Sep. 1999.
Game Enthusiast Online Highlights, Mar. 18, 1999.
Game Enthusiast Online Highlights, Mar. 19, 1999.
Game Enthusiast Online Highlights, Mar. 17, 1999.
Game Enthusiast Online Highlights, Oct. 20, 1999.
Joel Easley, "PlayStation II Revealed", Game Week, Sep. 29, 1999.
Inside Sony's Next Generation Playstation, ©1999.
Press Releases, Mar. 18, 1999.
Chris Johnston, "PlayStation Part Deux", Press Start, ©1999.
Nikkei Shimbun, "Sony Making SME, Chemical and SPT into Wholly–Owned Subsidiaries", Mar. 9, 1999.
AM News: Japanese Developers Not All Sold on PS2, Next Generation, Mar. 16, 1999.
Sony To Turn PlayStation Maker Into Wholly Owned Unit—Nikkei, Dow Jones News Service, Mar. 8, 1999.
Yumiko Ono, Sony Antes Up Its Chips In Bet On New Game System, Dow Jones News Service, Mar. 4, 1999.
MacWeek.Com Gets Inside Story on Connectix VGS for Windows; Controversial Emulator of Sony PlayStation Games Cureently Available for Macs Only, Business Wire, Mar. 12, 1999.
"DexDrive Bridges Gap", The Tampa Tribune, Mar. 12, 1999.
A Microprocessor With a 128b CPU, 10 Floating–Point MAC's, 4 Floating–Point Dividers, and an MPEG2 Decoder, 1999 IEEE International Solid–State Circuits Conference, Feb. 16, 1999.
Dreamcast Instruction Manual, Sega Enterprises, Ltd., ©1998.
"Sega To Launch Video Camera for Dreamcast", Reuters Business News, Feb. 16, 2000.
David Pescovitz, "Dream On", Wired, Aug. 1999.
Randy Nelson, "Dreamcast 101: Everything You Ever Wanted To Know About Sega's Powerful New Console", Official Sega Dreamcast Magazine, Jun. 1999.
2D/3D Graphics Card User Manual, Guillemot ©1999.
Nintendo 64 Instruction Booklet, Nintendo of America, 1998.
Steven Levy, "Here Comes PlayStation II", Newsweek, Mar. 6, 2000.
David Sheff, "Sony Smackage: Test Driving The PlayStation II", Wired, Nov. 1999.
Introducing The Next Generation PlayStation, Sony Computer Entertainment Inc., ©1999.
Leadtek GTS, Aug. 3, 2000, www.hexus.net.
Voodoo 5 5500 Review, Jul. 26, 2000, www.hexus.net.
ATI Radeon 64 Meg DDR OEM, Aug., 19, 2000, www.hexus.net.
Microsoft Xbox—The Future of Gaming, Microsoft Xbox Performance Sheet, www.xbox.com.
Robert L. Cook, "Shade Trees", Computer Graphics, vol. 18, No. 3, Jul. 1984.
Wang et al., "Second–Depth Shadow Mapping", Department of Computer Science, Univ. N.C, Chapel Hill, N.C. pp. 1–7.
Peercy et al., "Efficient Bump Mapping Hardware", Computer Graphics Proceedings, Annual Conference Series, 1997.
Gustavo Oliveira, "Refractive Texture Mappig, Part One", www.gamasutra.com, Nov. 10, 2000.
John Schlag, Fast Embossing Effects on Raster Image Data, Graphics Gems IV, Edited by Paul S. Heckbert, Computer Science Department, Carnegie Mellon University, Academic Press, Inc., 1994, pp. 433–437.
James F. Blinn, "Simulationof Wrinkled Surfaces," Caltech/JPL, pp. 286–292, SIGGRAPH 78 (1978).
Tomas Möller and Eric Haines "Real–Time Rendering", AK Peters, Ltd., ©1999, pp. 127–142.
Technical Presentation: Vertex Buffers, posted Jun. 12, 2000, www.nvidia.com.
Technical Presentation: Hardware Transform and Lighting, www.nvidia.com, posted Jun. 12, 2000.
Technical Presentation: Hardware Bump–mapping Choices and Concepts, Jun. 7, 2000, www.nvidia.com.
Technical Presentation: How to Bump Map a Skinned Polygonal Model, Jun. 7, 2000, www.nvidia.com.
Technical Presentation: Computations for Hardware Lighting and Shading, Mar. 17, 2000, www.nvidia.com.
McCool, M.D. et al., "Texture Shaders," Proceedings 1999 Eurographics/SIGGRAPH Workshop on Graphics Hardware, Los Angeles, CA, Aug. 8–9, 1999, SIGGRAPH/Eurographics Workshop on Graphics Hardware, New York, NY: ACM, US, pp. 122–125, Figure 4 (Aug. 8, 1999).
Heidrich W. et al., "Illuminating micro geometry based on precomputed visibility," Computer Graphics Proceedings, SIGGRAPH 2000, Conference Proceedings, New Orleans, LA, USA, Jul. 27–28, 2000, pp. 455–464, New York, NY, USA, ACM.
Peter J. Kovach, Inside Direct3D pp. 249–262, Microsoft Press 2000.
GDC 2000: Advanced OpenGL Game Development "A Practical and Robust Bump–mapping Technique for Today's GPUs", by Mark Kilgard, Jul. 5, 2000, www.nvidia.com.
Technical Presentations: "Texture Space Bump Mapping", Sim Dietrich, Nov. 10, 2000, www.nvidia.com.
Technical Presentation: Practical Bump–mapping for Today's GPUs, Mar. 17, 2000 www.nvidia.com.
Technical Presentation: Shadows, Transparency, & Fog, Mar. 17, 2000 www.nvidia.com.

Technical Presentation: GeForce 256 Register Combines, Mar. 17, 2000,www.nvidia.com.
Technical Presentation: TexGen & The Texture Matrix, Mar. 15, 2000, www.nvidia.com.
Technical Presentation: Toon Shading, Mar. 15, 2000, www.nvidia.com.
Technical Presentation: D3D 7 Vertex Lighting, Mar. 15, 2000, www.nvidia.com.
Technical Presentation: Per–Pixel Lighting (by S. Dietrich) Mar. 14, 2000 www.nvidia.com.
Technical Presentation: GeForce 256 and RIVA TNT Combiners, Dec. 8, 1999, www.nvidia.com.
Technical Presentation: Vertex Cache Optimization, Nov. 12, 1999, www.nvidia.com.
Technical Presentation: Vertex Blending, Nov. 12, 1999, www.nvidia.com.
Technical Presentation: Hardware Transform and Lighting, Nov. 12, 1999, www.nvidia.com.
Technical Presentation: GeForce 256 Overview, Nov. 12, 1999, www.nvidia.com.
Technical Presentation: DirectX 7 and Texture Management, Nov. 12, 1999 www.nvidia.com.
Technical Presentation: Dot Product Lighting, Nov. 12, 1999, www.nvidia.com.
Technical Presentation: Texture Coordinate Generation, Nov. 3, 1999, www.nvidia.com.
Technical Presentation: Phong Shading and Lightmaps, Nov. 3, 1999, www.nvidia.com.
Technical Presentation: The ARB_multitexture Extension, Nov. 3, 1999, www.nvidia.com.
Technical Presentation: Multitexture Combiners, Nov. 3, 1999, www.nvidia.com.
Technical Presentation: Emboss Bump Mapping, Nov. 3, 1999, www.nvidia.com.
Technical Presentation: Hardware Accelerated Anisotropic Lighting, Nov. 3, 1999 www.nvidia.com.
Technical Presentation: Guard Band Clipping, Nov. 3, 1999, www.nvidia.com.
The RenderMan Interface, Stephan R. Keith, Version 3.1, Pixar Animation Studios, Sep. 1989.
The RenderMan Interface, Version 3.2, Pixar Animation Studios, Jul. 2000, www.pixar.com.
NVIDIA Product Overview, "GeForce2Ultra", NVIDIA Corporation, Aug. 21, 2000, www.nvidia.com.
Duke, "Dreamcast Technical Specs", Sega Dreamcast Review, Sega, 2/99, www.game–revolution.com.
Marlin Rowley, "GeForce 1 & 2 GPU Speed Tests", May 11, 2000, www.g256.com.
"Dreamcast: The Full Story", Next Generation, Sep. 1998.
DirectX 7.0 Programmer's Reference, Microsoft Corporation,1995–1999 (as part of the DirectX 7.0 SDK on the Companion CD included with the "Inside Direct3D", Microsoft Programming Series, Peter J. Kovach, Microsoft Press, 1999).
"Inside Direct3D", Microsoft Programming Series, Peter J. Kovach, Microsoft Press, 1999.
"OpenGL Programming Guide, The Official Guide to Learning OpenGL, Release 1", Jackie Nieder, Tom David, Mason Woo, Addision–Wesley Publishing Co., 1993.
"Procedural Elements for Computer Graphics," Second Edition, David F. Rogers, McGraw Hill, 1998.
"Real–Time Rendering," Tomas Molleir, Eric Haines, AK Peters, 1999.
"Computer Graphics, Principles and Practice," Second Edition, The Systems Programming Series, Foley, van Dam, Fiener, Hughes, Addison Wesley, 1990.
"Principles of Three–Dimensional Computer Animation", Revised Edition, Michael O'Rourke, W.W. Norton & Company, 1998.

* cited by examiner

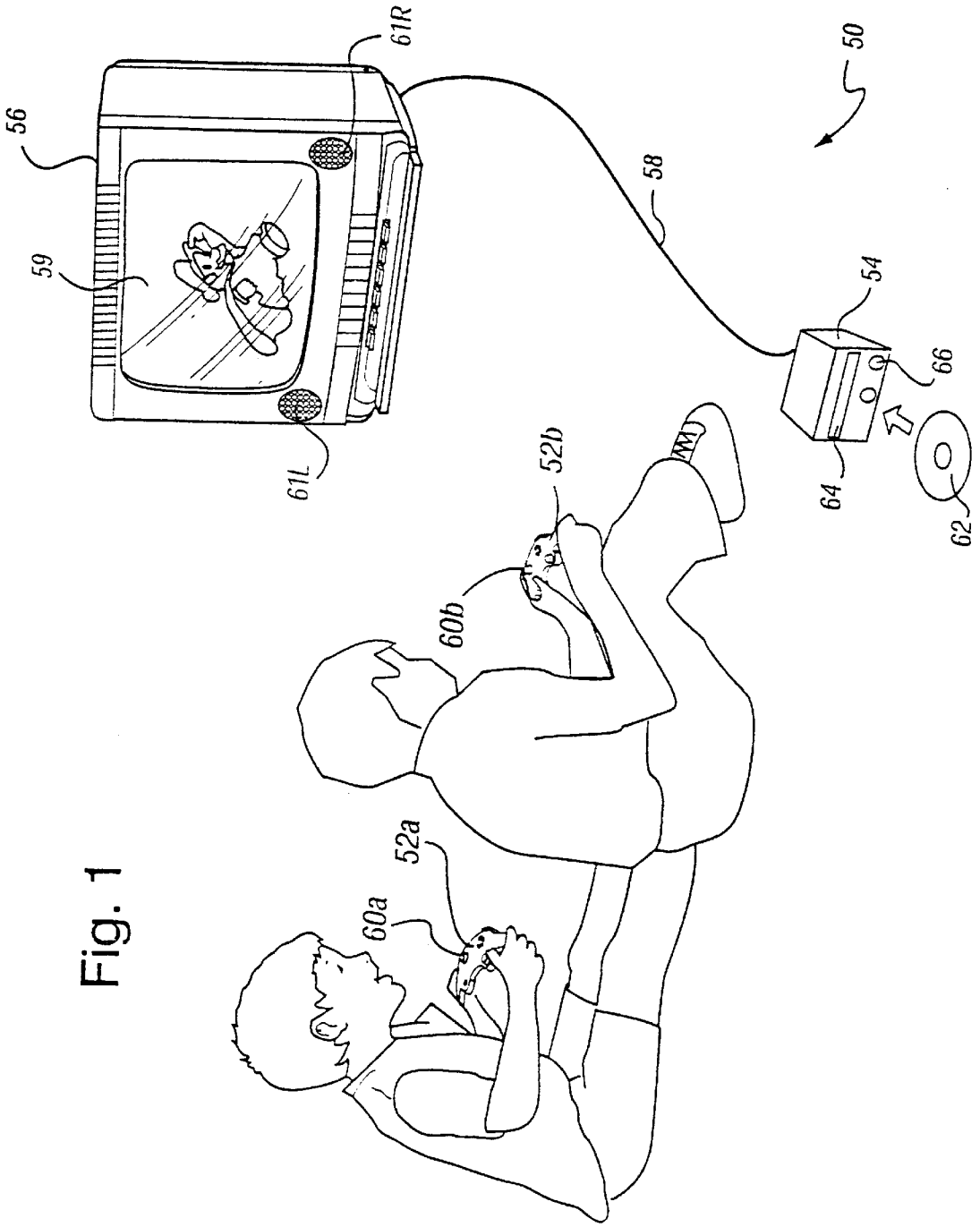


Fig. 1

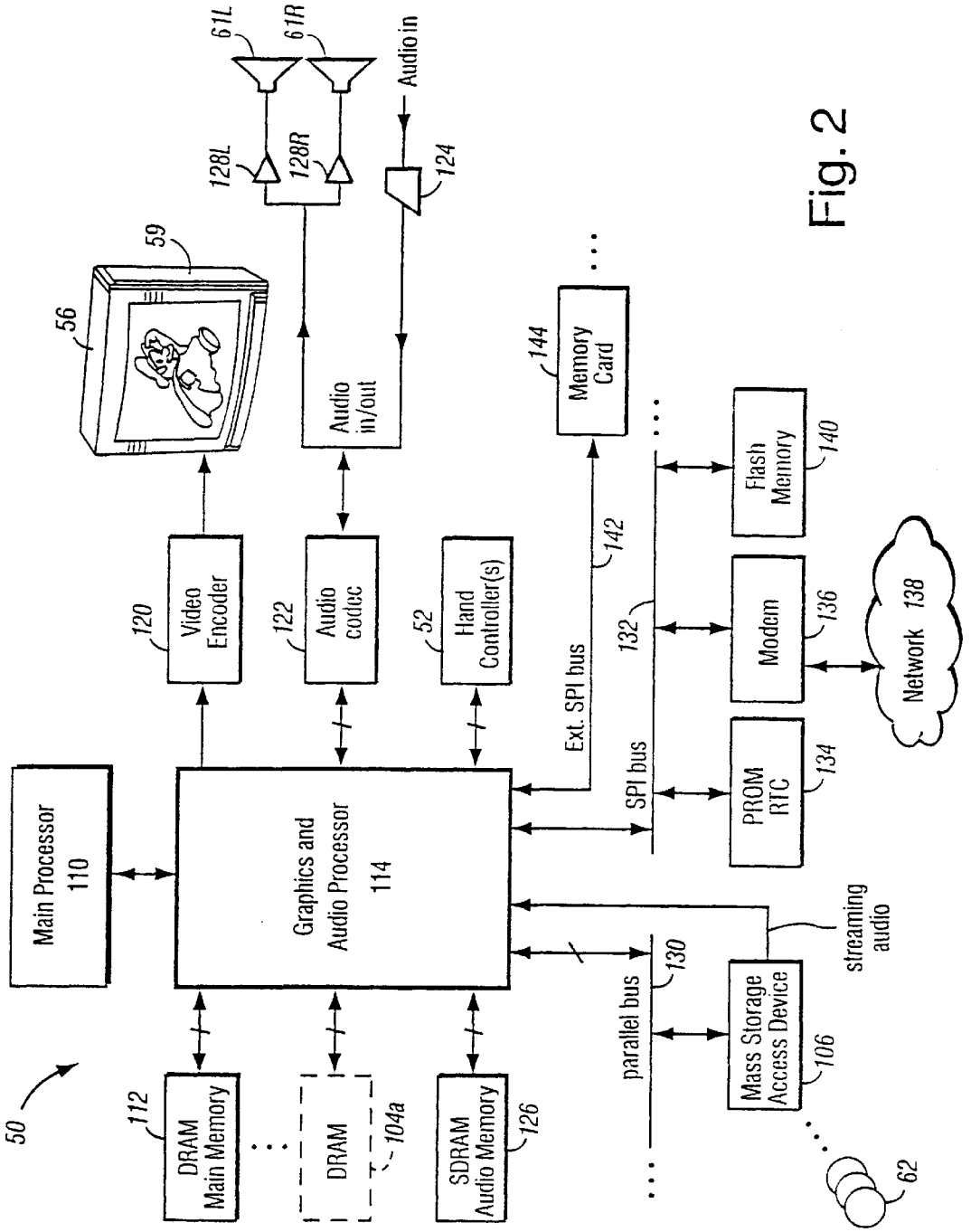

50
Main Processor 110
Graphics and Audio Processor 114
DRAM Main Memory
112
DRAM
104a
SDRAM Audio Memory
126
Video Encoder
120
Audio codec
122
Hand Controller(s)
52
56
59
128L
128R
61L
61R
Audio in/out
Audio in
124
Ext. SPI bus
142
Memory Card
144
SPI bus
132
PROM RTC
134
Modem
136
Flash Memory
140
Network 138
parallel bus
130
Mass Storage Access Device
106
streaming audio
62

Fig. 2

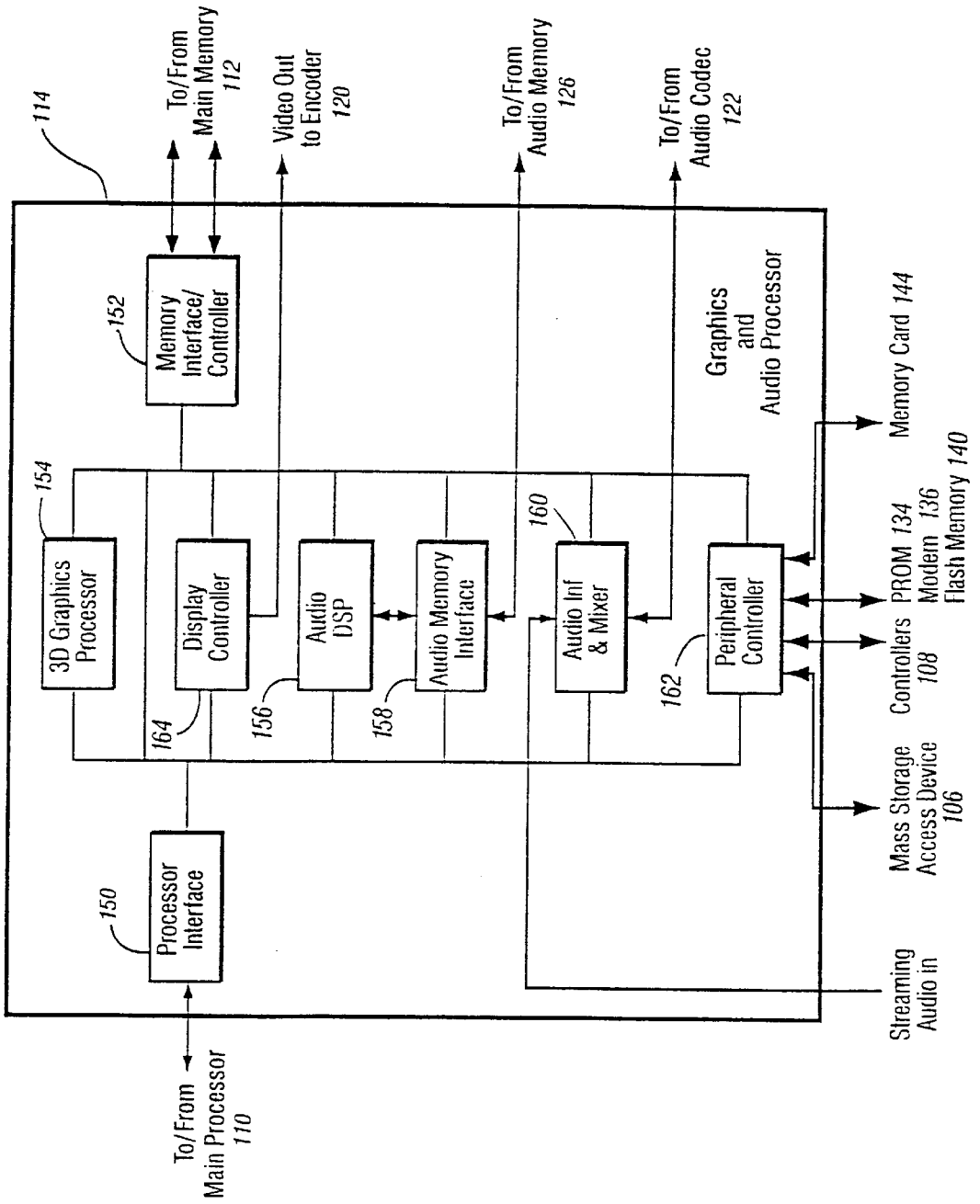

114
To/From Main Memory 112
Video Out to Encoder 120
To/From Audio Memory 126
To/From Audio Codec 122
152
Memory Interface/ Controller
Graphics and Audio Processor
Memory Card 144
154
3D Graphics Processor
Display Controller
Audio DSP
Audio Memory Interface
160
Audio Inf & Mixer
Peripheral Controller
PROM 134
Modem 136
Flash Memory 140
Controllers 108
164
156
158
162
Mass Storage Access Device 106
150
Processor Interface
Streaming Audio in
To/From Main Processor 110

Fig. 3

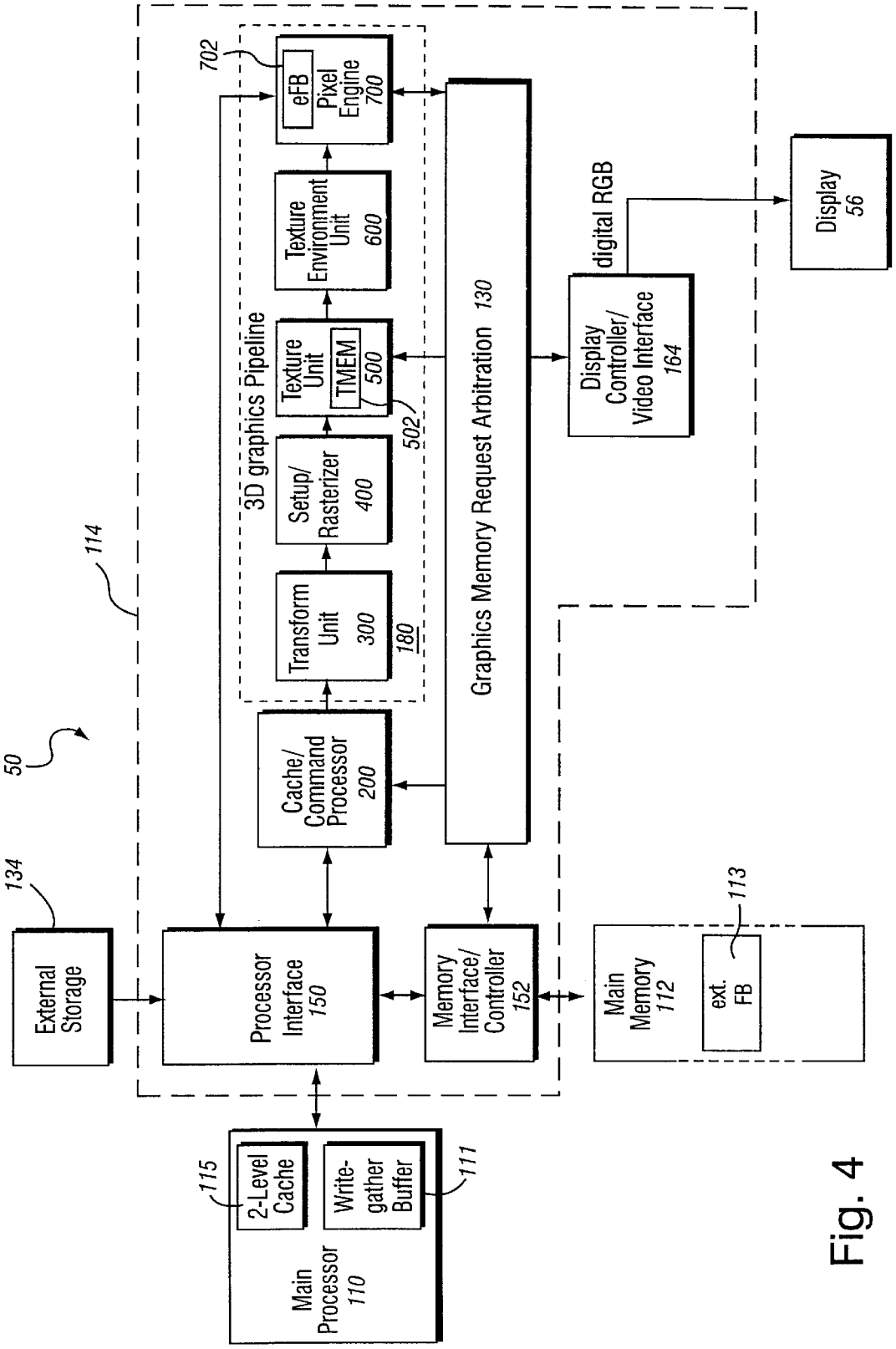

50
114
134
External Storage
Processor Interface 150
Cache/ Command Processor 200
3D graphics Pipeline
Transform Unit 300
180
Setup/ Rasterizer 400
Texture Unit
TMEM
500
502
Texture Environment Unit 600
702
eFB
Pixel Engine 700
Graphics Memory Request Arbitration 130
Memory Interface/ Controller 152
Main Memory 112
ext. FB
113
Display Controller/ Video Interface 164
digital RGB
Display 56
Main Processor 110
2-Level Cache
115
Write-gather Buffer
111

Fig. 4

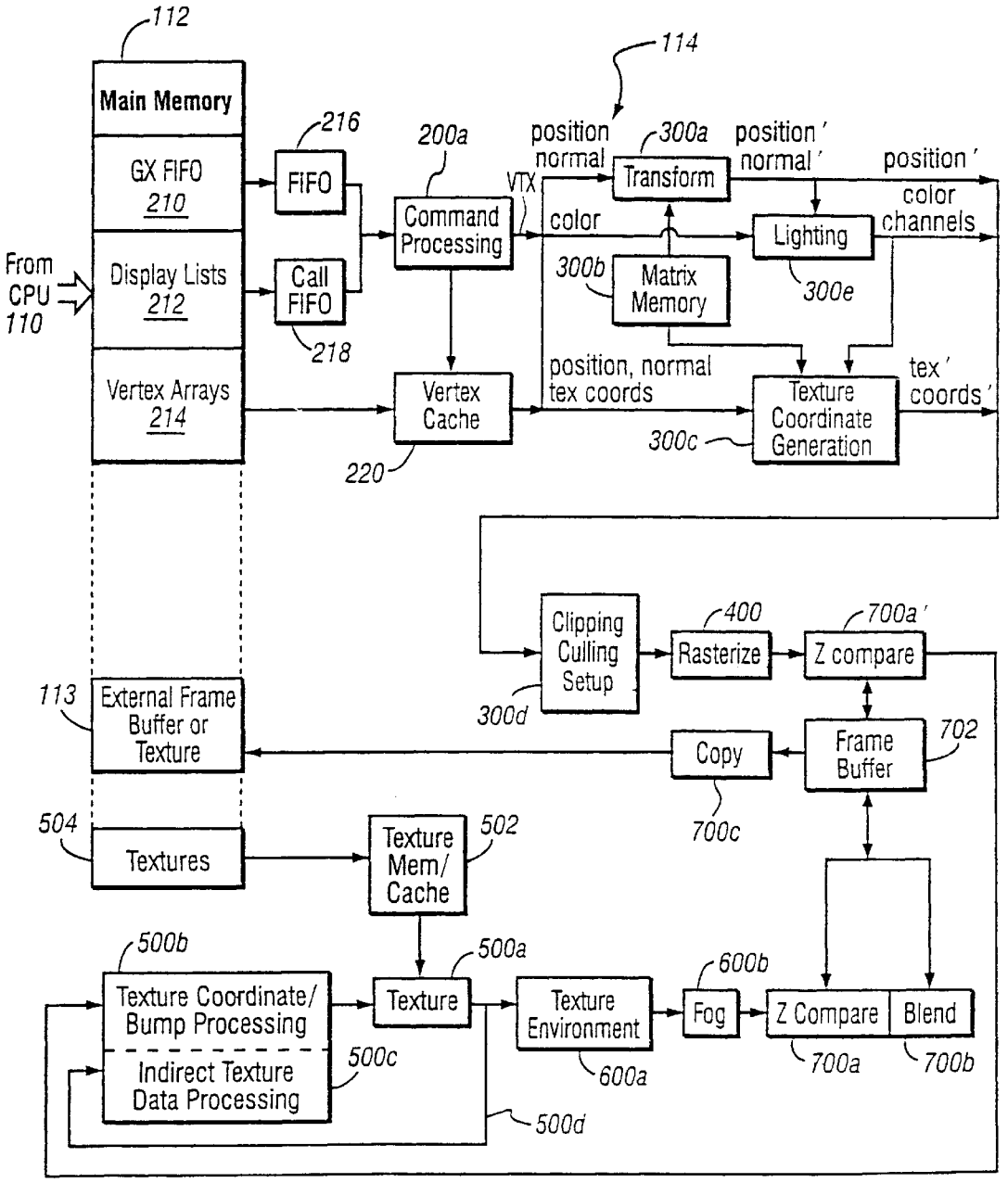
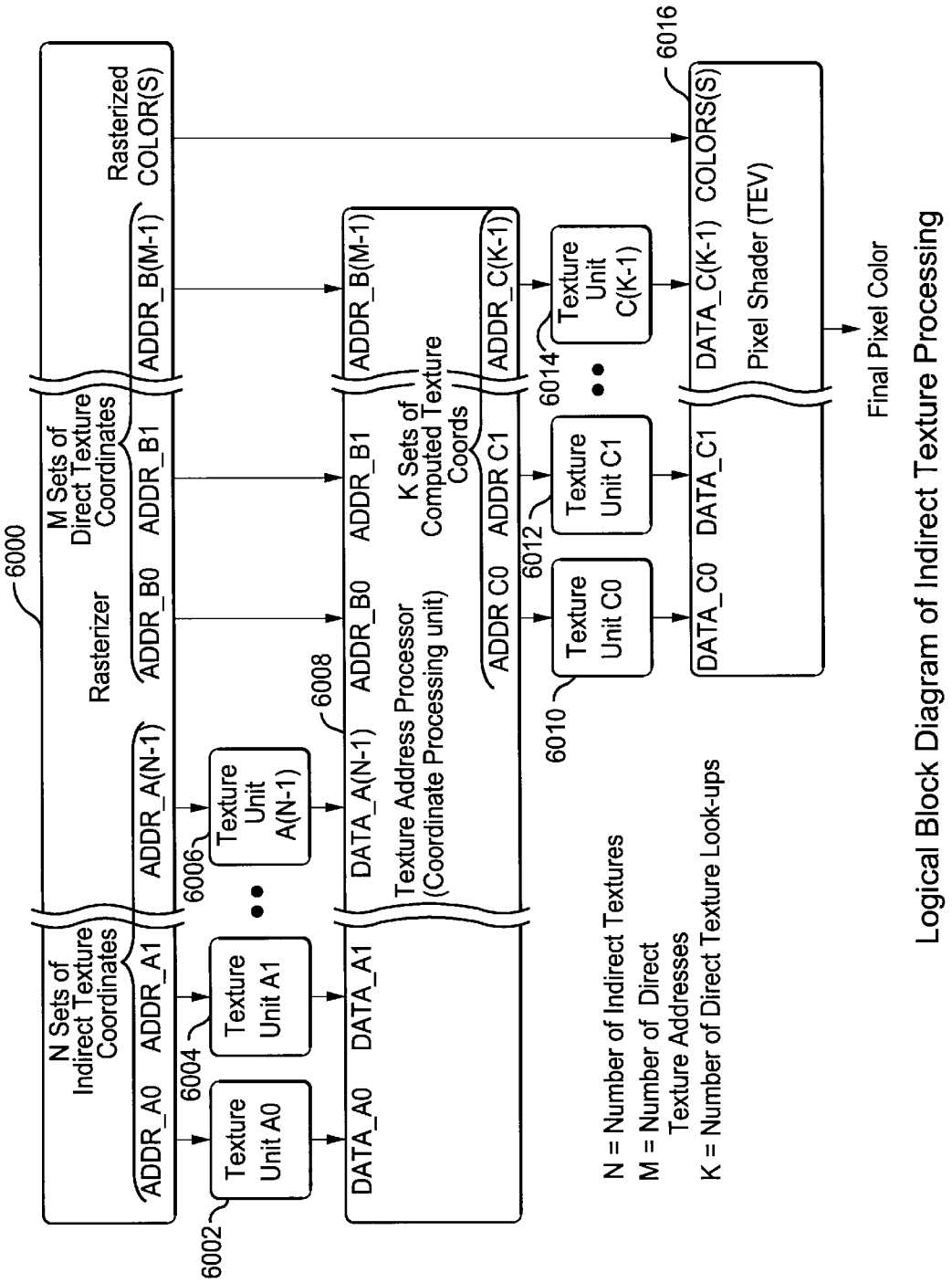


Fig. 5 EXAMPLE GRAPHICS PROCESSOR FLOW



Logical Block Diagram of Indirect Texture Processing

Fig. 6

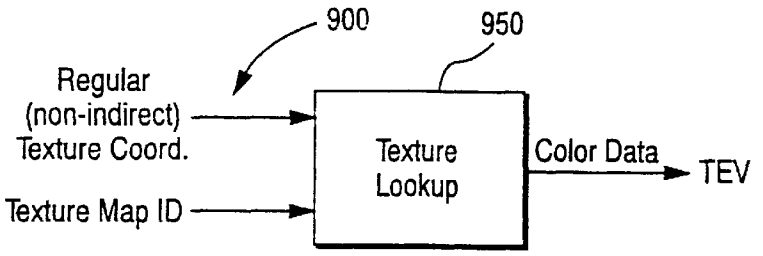


Fig. 7A

REGULAR TEXTURE LOOKUP

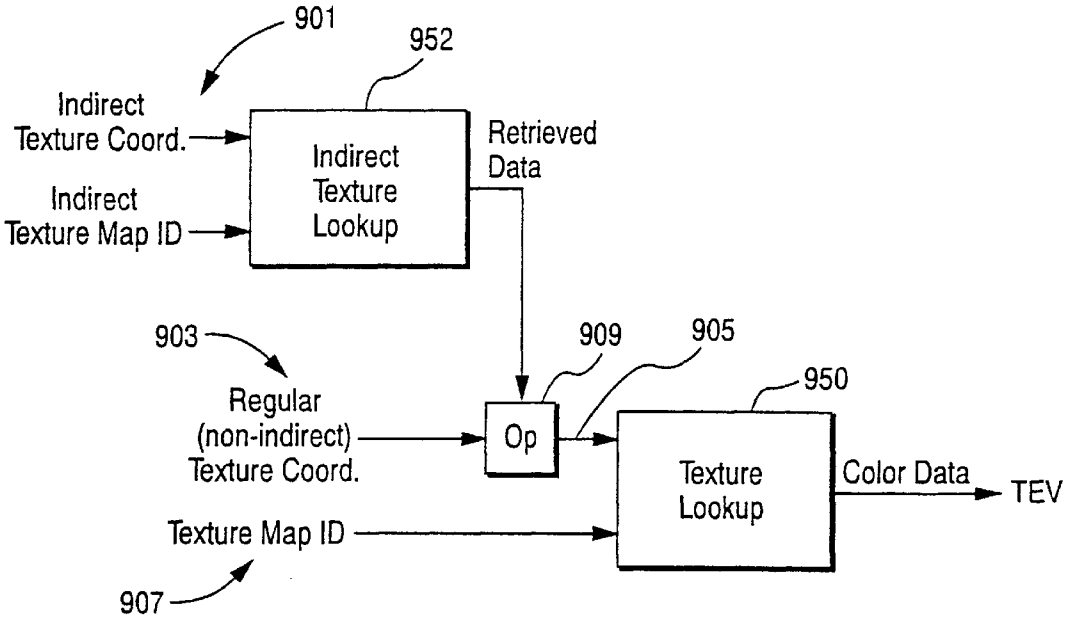
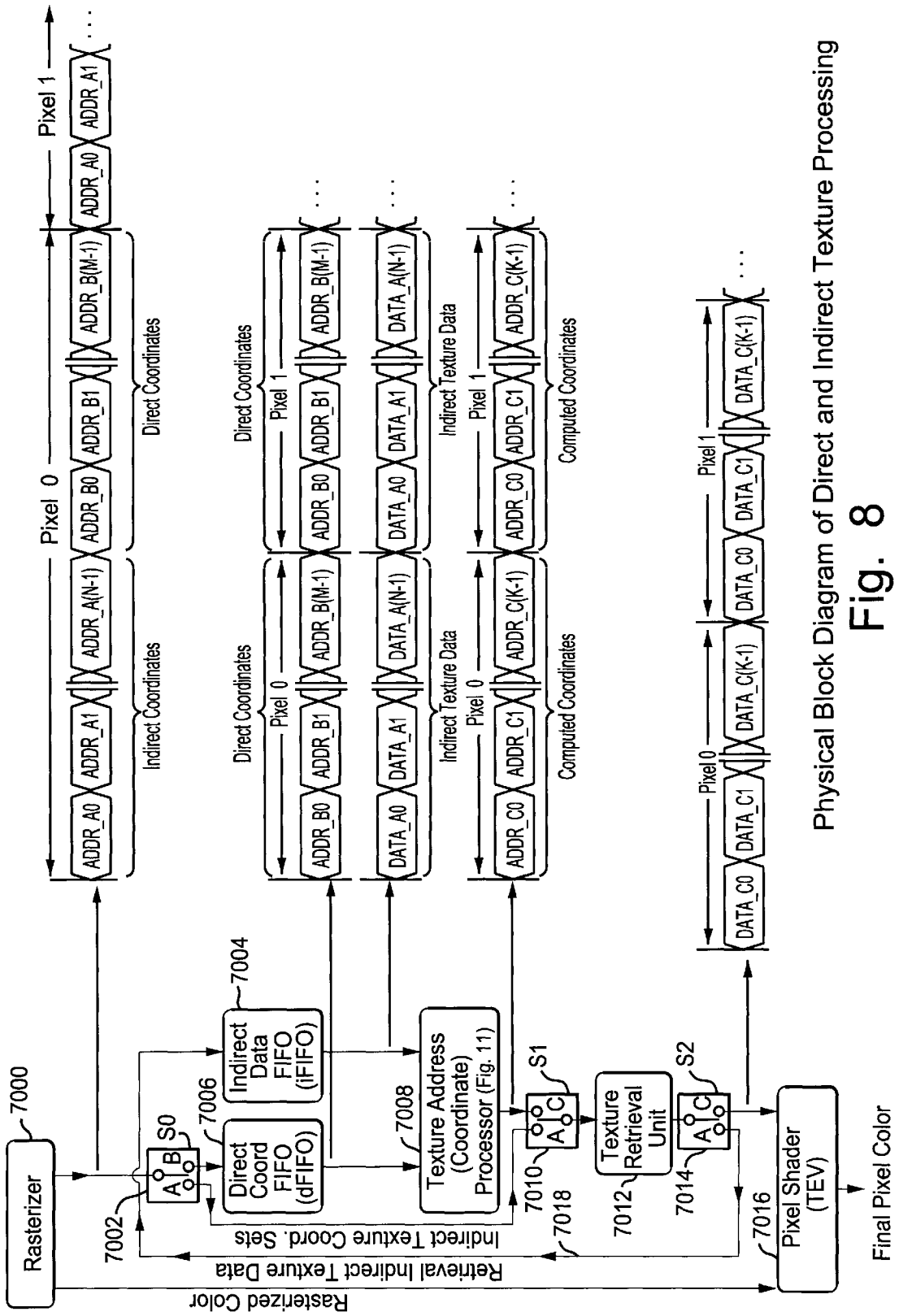


Fig. 7B

INDIRECT TEXTURE LOOKUP



Physical Block Diagram of Direct and Indirect Texture Processing

Fig. 8

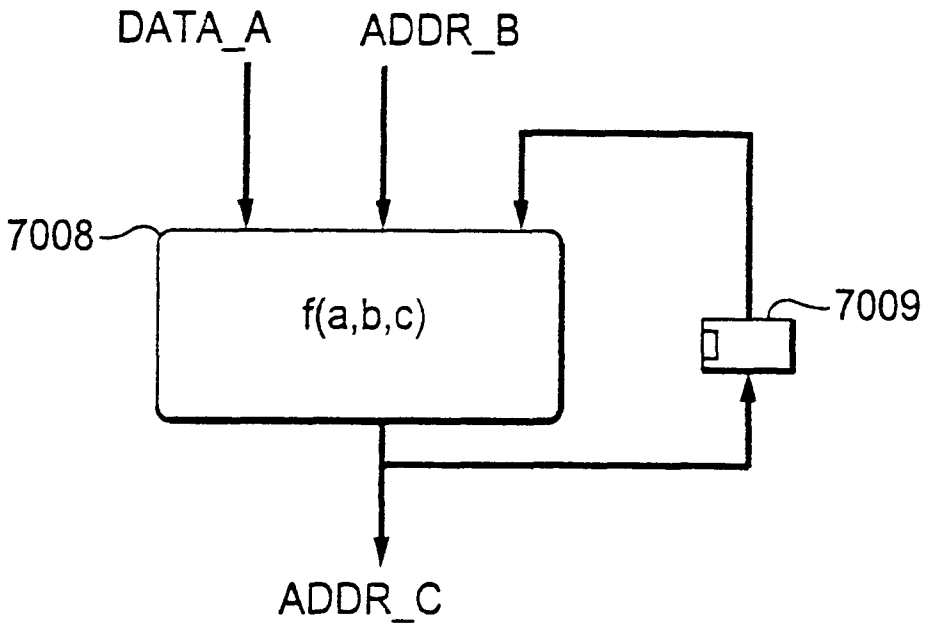


Fig. 9

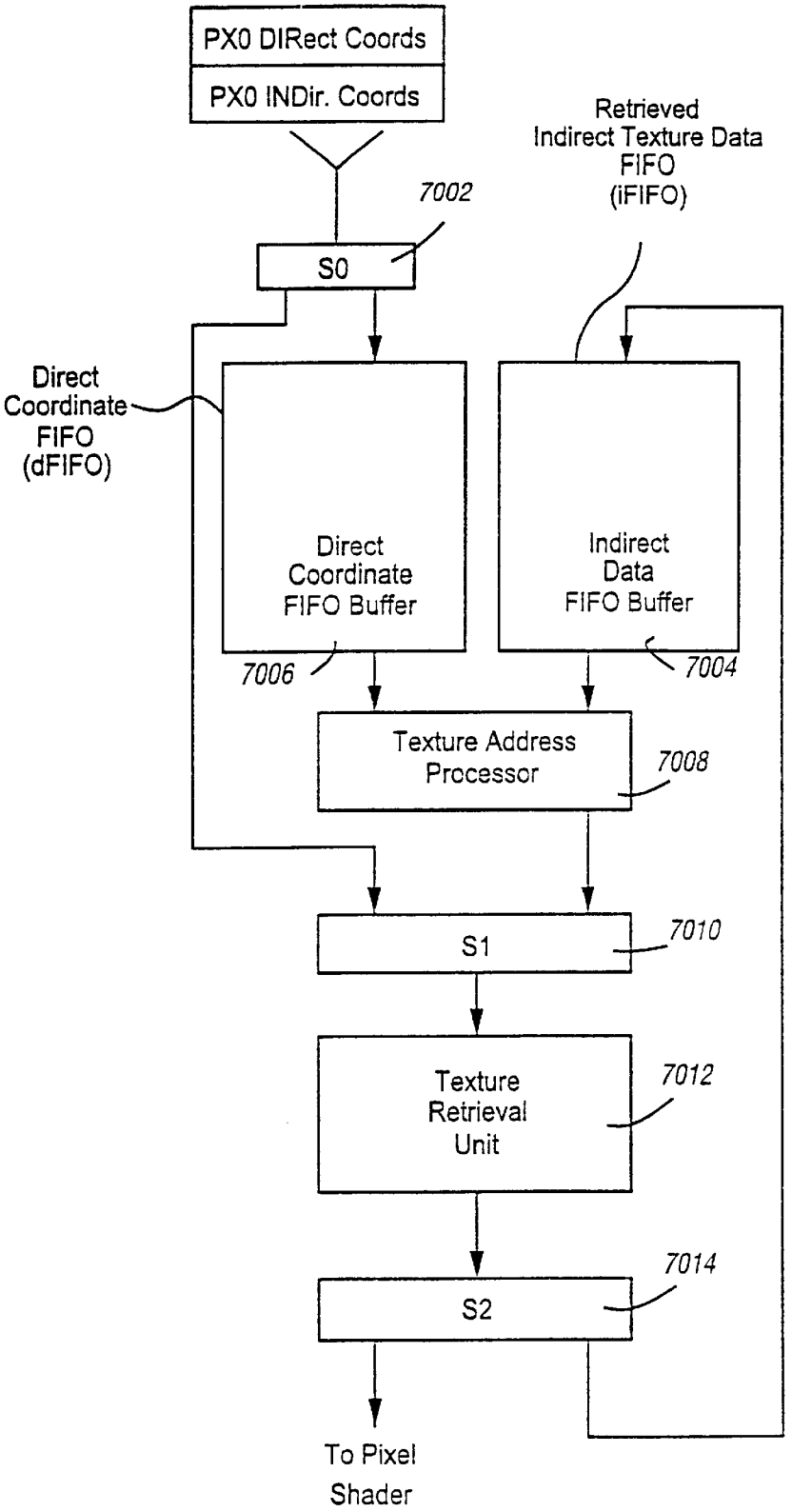

PX0 DIRect Coords
PX0 INDir. Coords
Retrieved
Indirect Texture Data
FIFO
(iFIFO)
7002
S0
Direct
Coordinate
FIFO
(dFIFO)
Direct
Coordinate
FIFO Buffer
Indirect
Data
FIFO Buffer
7006
7004
Texture Address
Processor
7008
S1
7010
Texture
Retrieval
Unit
7012
7014
S2
To Pixel
Shader

Fig. 10A

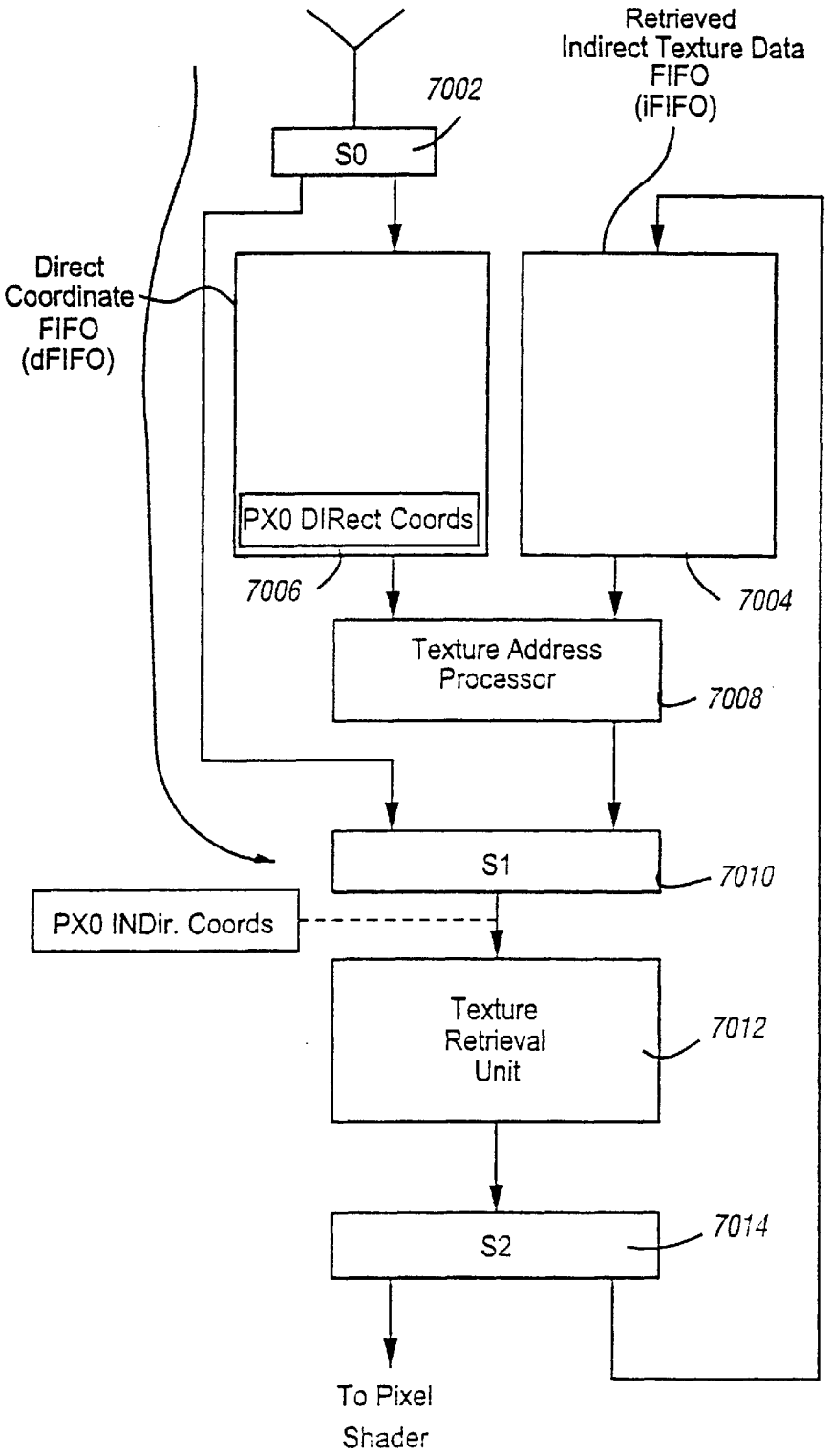

Retrieved
Indirect Texture Data
FIFO
(iFIFO)
7002
S0
Direct
Coordinate
FIFO
(dFIFO)
PX0 DIRect Coords
7006
7004
Texture Address
Processor
7008
S1
7010
PX0 INDir. Coords
Texture
Retrieval
Unit
7012
S2
7014
To Pixel
Shader

Fig. 10B

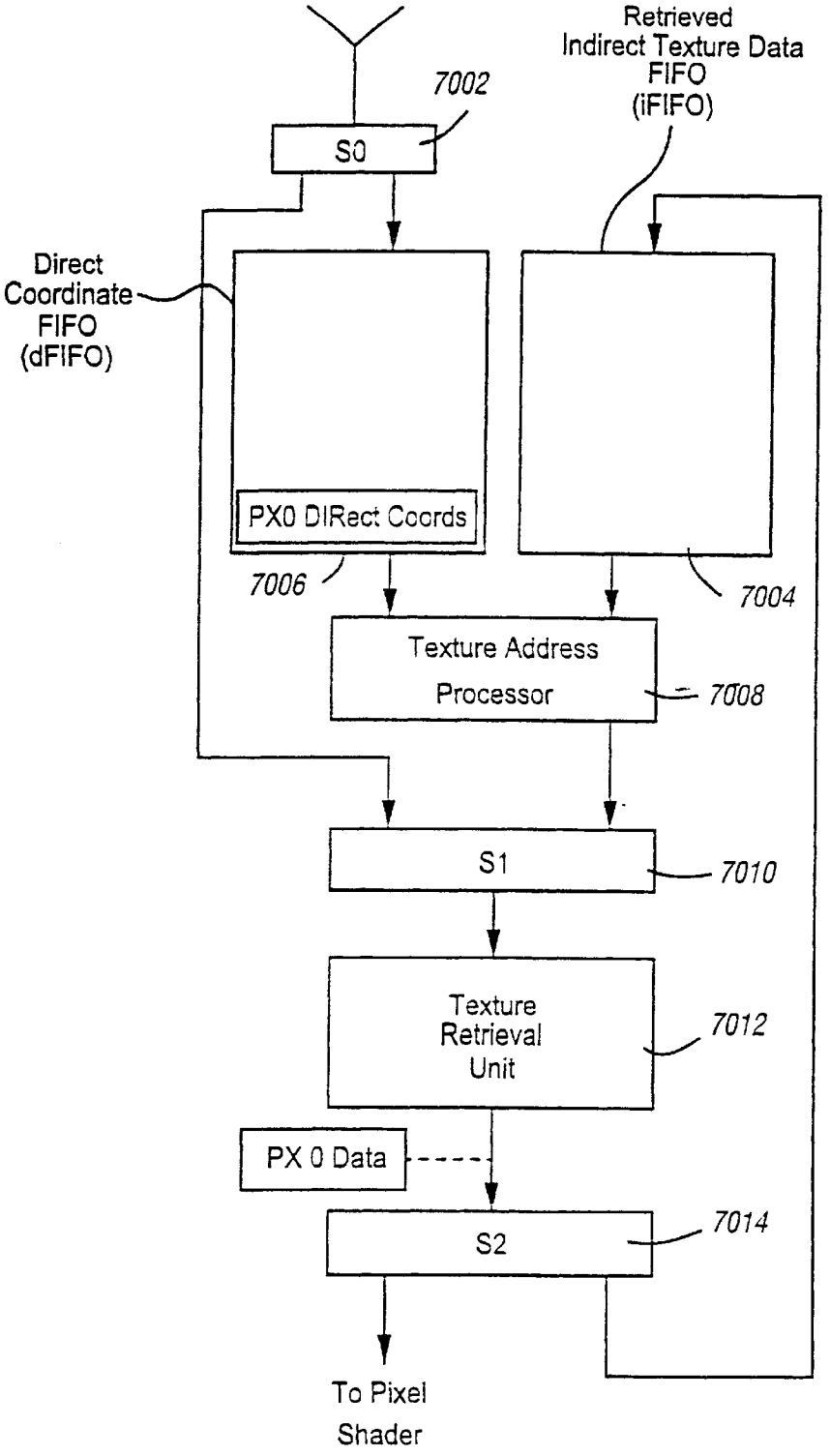

Retrieved
Indirect Texture Data
FIFO
(iFIFO)
7002
S0
Direct
Coordinate
FIFO
(dFIFO)
PX0 DIRect Coords
7006
7004
Texture Address
Processor
7008
S1
7010
Texture
Retrieval
Unit
7012
PX 0 Data
S2
7014
To Pixel
Shader

Fig. 10C

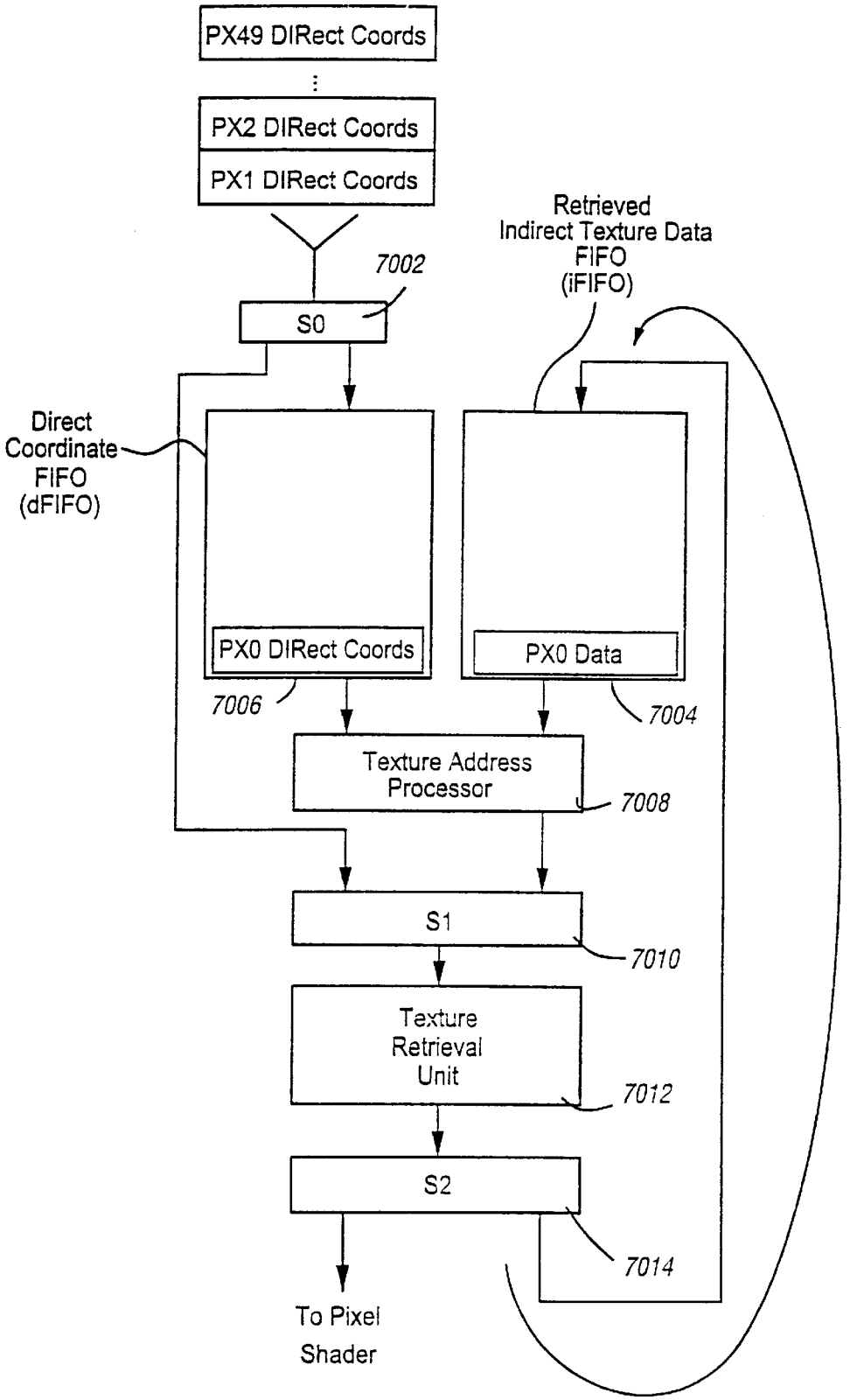

PX49 DIRect Coords
PX2 DIRect Coords
PX1 DIRect Coords
7002
S0
Retrieved
Indirect Texture Data
FIFO
(iFIFO)
Direct
Coordinate
FIFO
(dFIFO)
PX0 DIRect Coords
PX0 Data
7006
7004
Texture Address
Processor
7008
S1
7010
Texture
Retrieval
Unit
7012
S2
7014
To Pixel
Shader

Fig. 10D

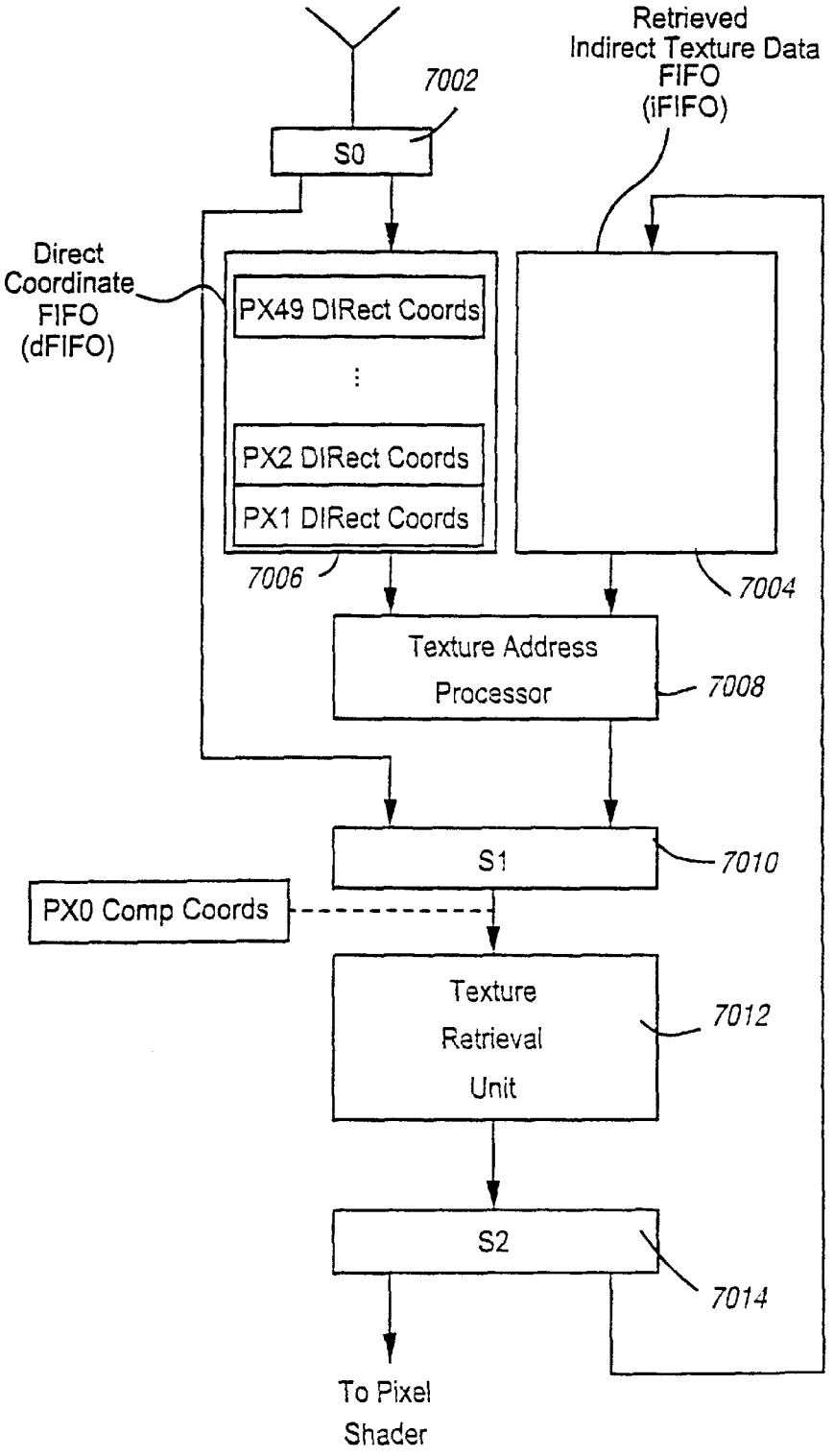

Retrieved
Indirect Texture Data
FIFO
(iFIFO)
7002
S0
Direct
Coordinate
FIFO
(dFIFO)
PX49 DIRect Coords
PX2 DIRect Coords
PX1 DIRect Coords
7006
7004
Texture Address
Processor
7008
S1
7010
PX0 Comp Coords
Texture
Retrieval
Unit
7012
S2
7014
To Pixel
Shader

Fig. 10E

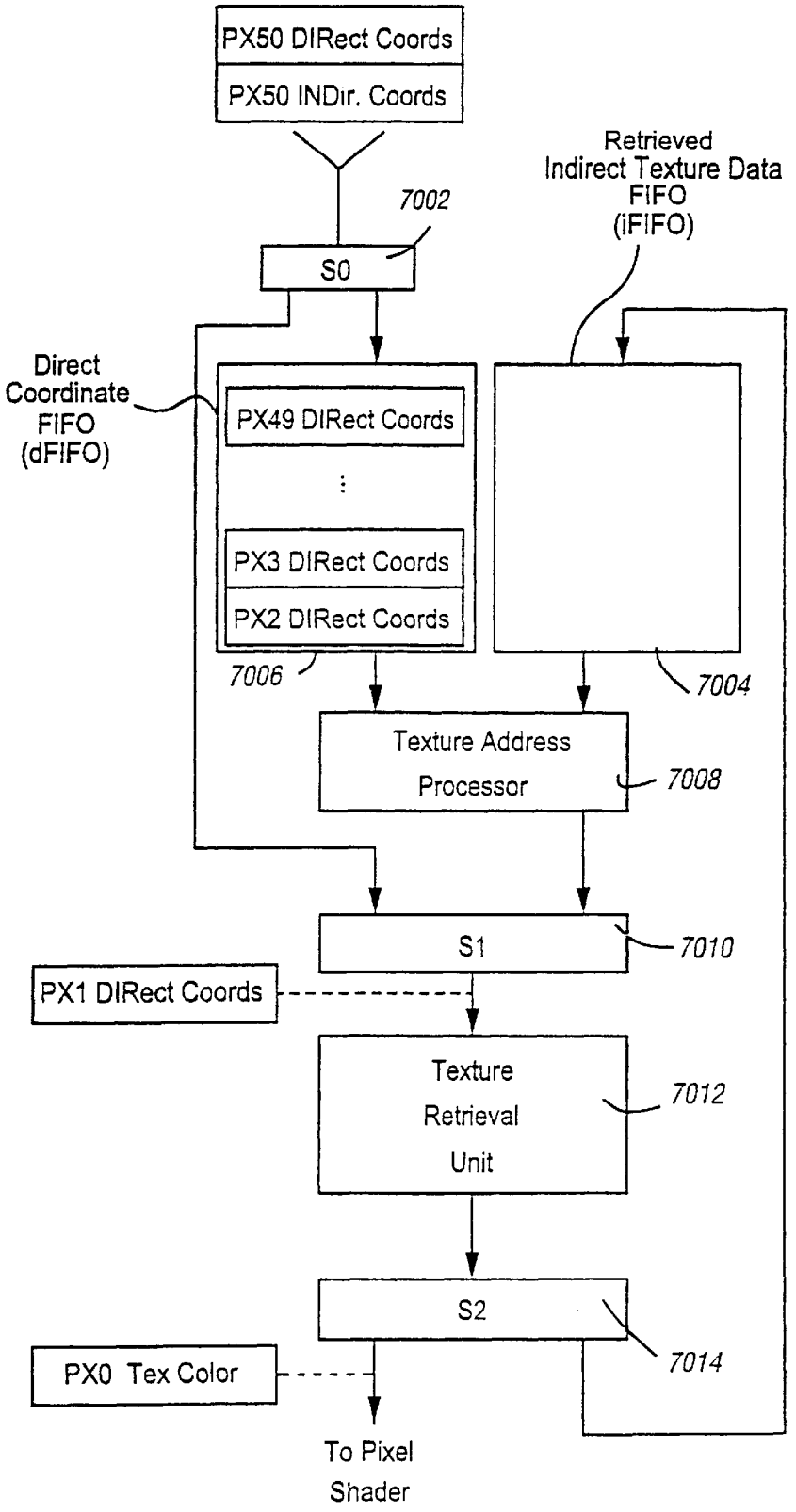

PX50 DIRect Coords
PX50 INDir. Coords
7002
S0
Retrieved
Indirect Texture Data
FIFO
(iFIFO)
Direct
Coordinate
FIFO
(dFIFO)
PX49 DIRect Coords
PX3 DIRect Coords
PX2 DIRect Coords
7006
7004
Texture Address
Processor
7008
S1
7010
PX1 DIRect Coords
Texture
Retrieval
Unit
7012
S2
7014
PX0 Tex Color
To Pixel
Shader

Fig. 10F

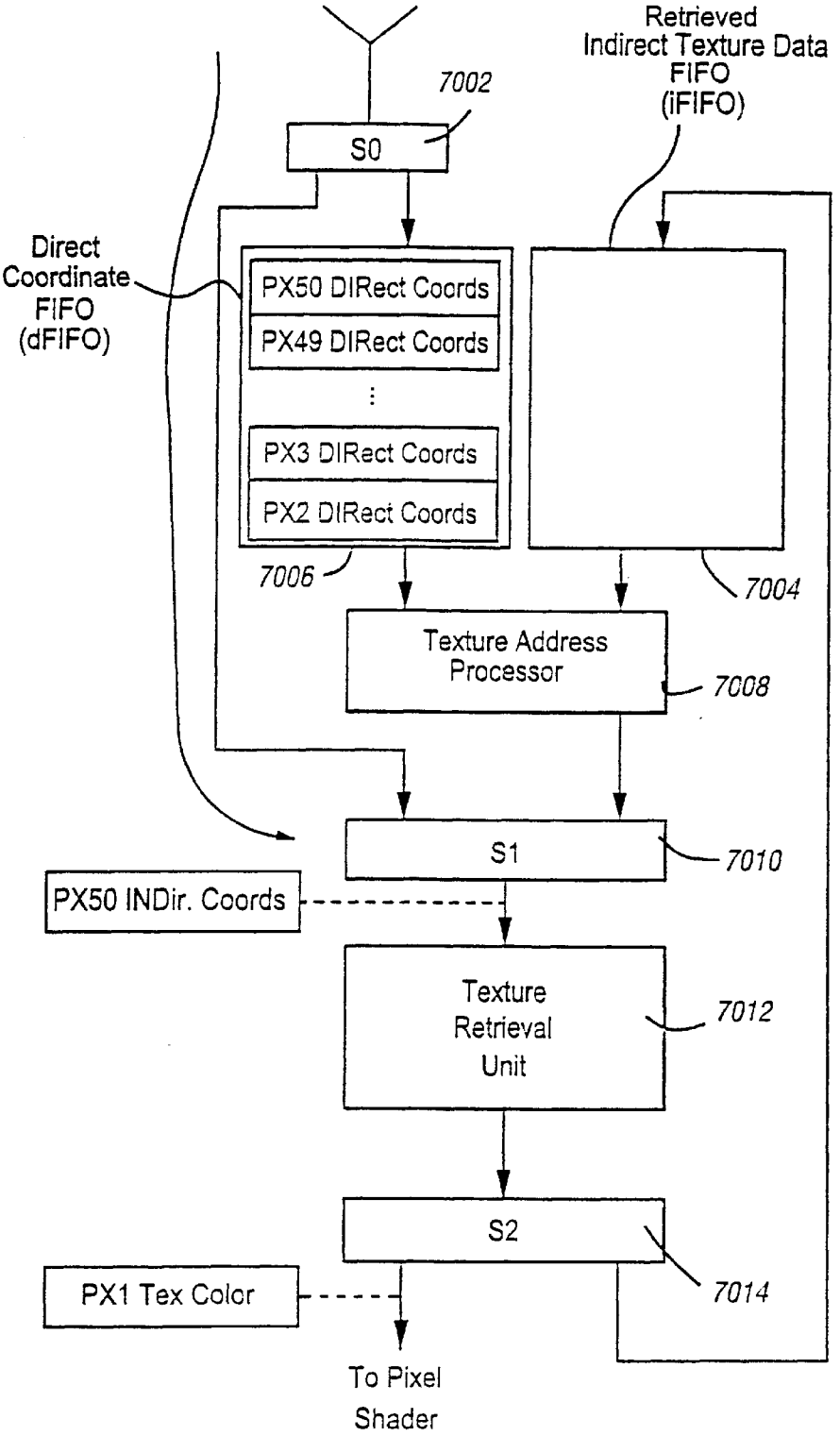

Retrieved
Indirect Texture Data
FIFO
(iFIFO)
7002
S0
Direct
Coordinate
FIFO
(dFIFO)
PX50 DIRect Coords
PX49 DIRect Coords
PX3 DIRect Coords
PX2 DIRect Coords
7006
7004
Texture Address
Processor
7008
S1
7010
PX50 INDir. Coords
Texture
Retrieval
Unit
7012
S2
7014
PX1 Tex Color
To Pixel
Shader

Fig. 10G

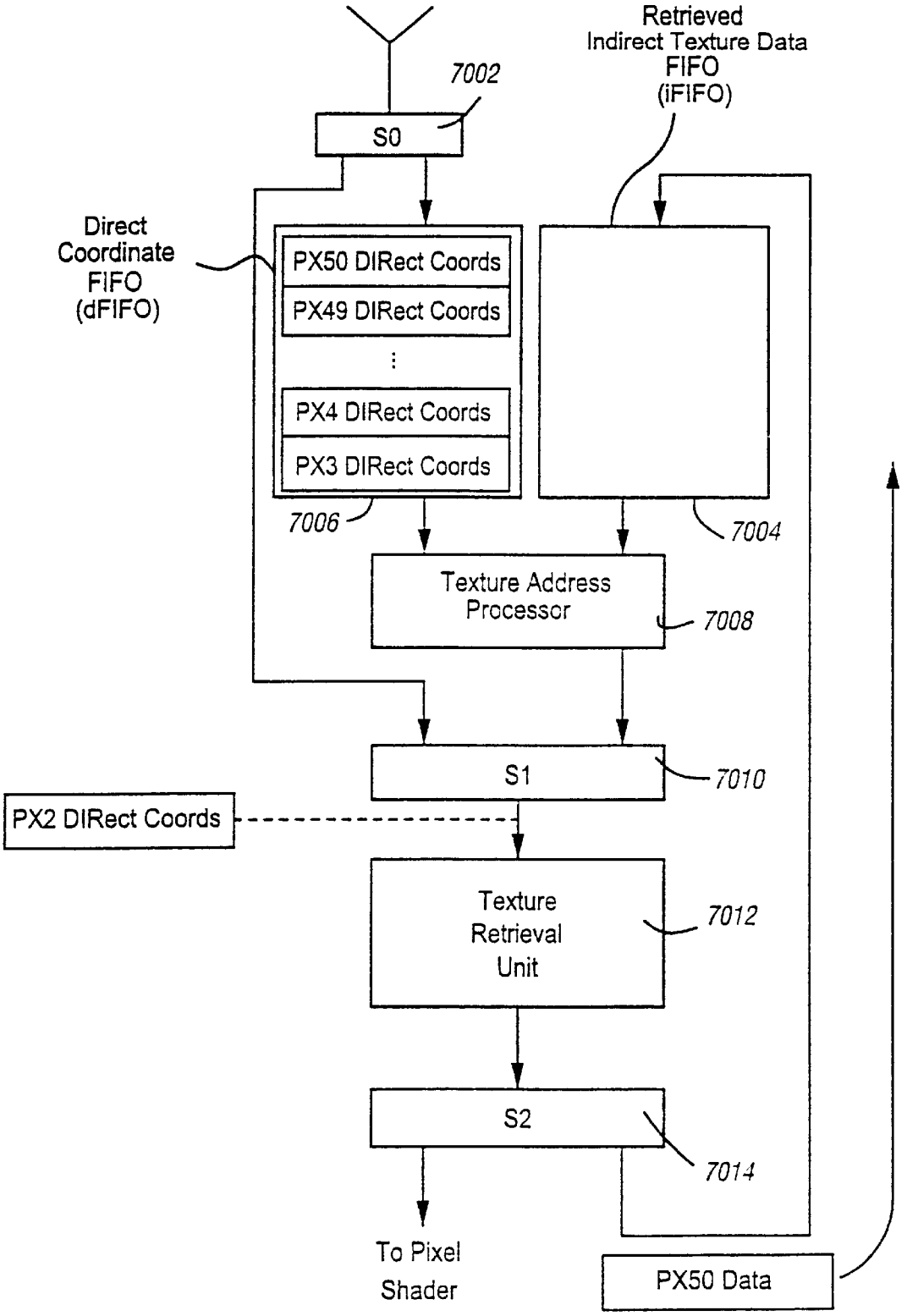

Retrieved
Indirect Texture Data
FIFO
(iFIFO)
7002
S0
Direct
Coordinate
FIFO
(dFIFO)
PX50 DIRect Coords
PX49 DIRect Coords
PX4 DIRect Coords
PX3 DIRect Coords
7006
7004
Texture Address
Processor
7008
S1
7010
PX2 DIRect Coords
Texture
Retrieval
Unit
7012
S2
7014
To Pixel
Shader
PX50 Data

Fig. 10H

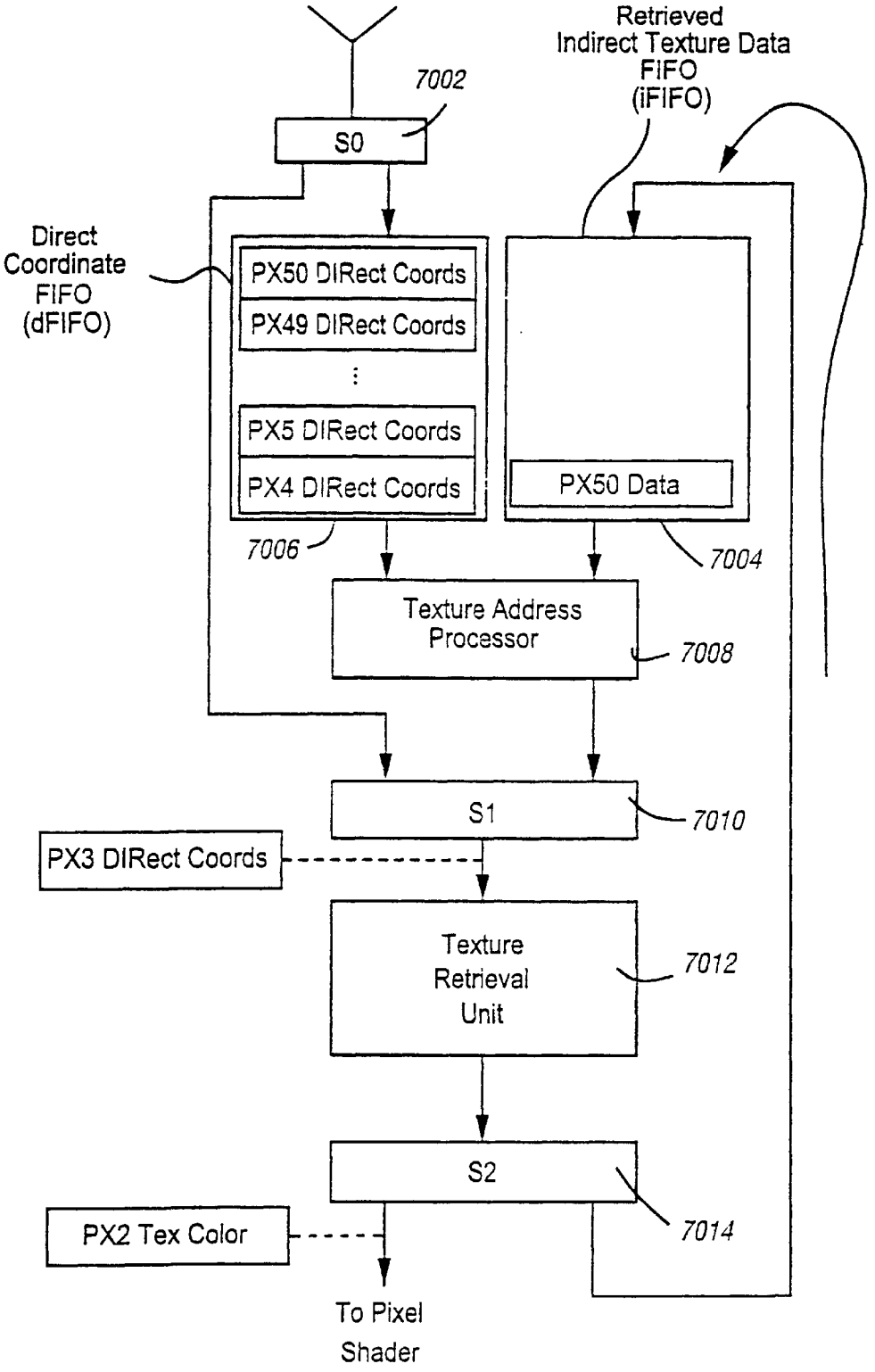

Retrieved
Indirect Texture Data
FIFO
(iFIFO)
7002
S0
Direct
Coordinate
FIFO
(dFIFO)
PX50 DIRect Coords
PX49 DIRect Coords
PX5 DIRect Coords
PX4 DIRect Coords
PX50 Data
7006
7004
Texture Address
Processor
7008
S1
7010
PX3 DIRect Coords
Texture
Retrieval
Unit
7012
S2
7014
PX2 Tex Color
To Pixel
Shader

Fig. 10I

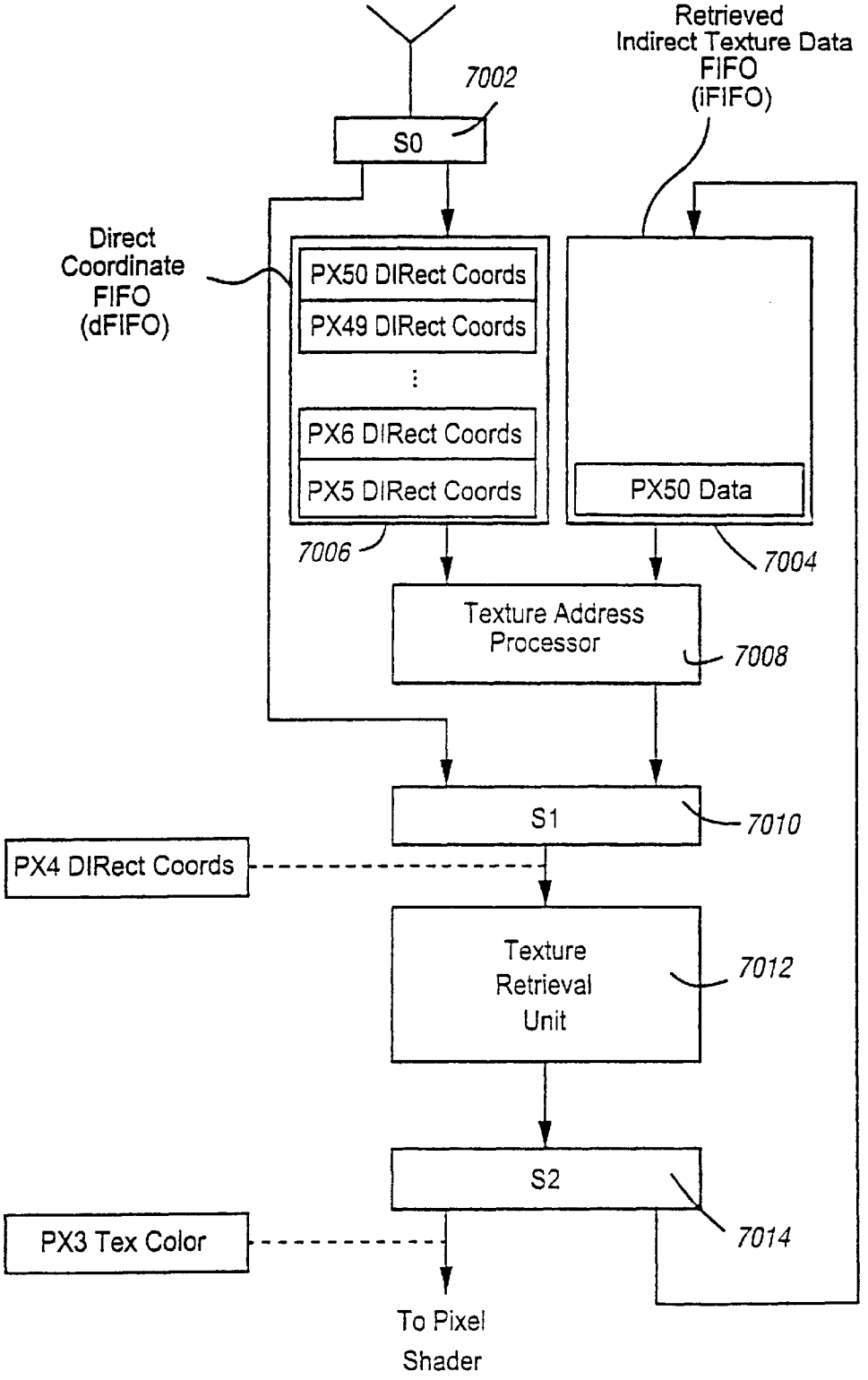

Retrieved
Indirect Texture Data
FIFO
(iFIFO)
7002
S0
Direct
Coordinate
FIFO
(dFIFO)
PX50 DIRect Coords
PX49 DIRect Coords
PX6 DIRect Coords
PX5 DIRect Coords
PX50 Data
7006
7004
Texture Address
Processor
7008
S1
7010
PX4 DIRect Coords
Texture
Retrieval
Unit
7012
S2
7014
PX3 Tex Color
To Pixel
Shader

Fig. 10J

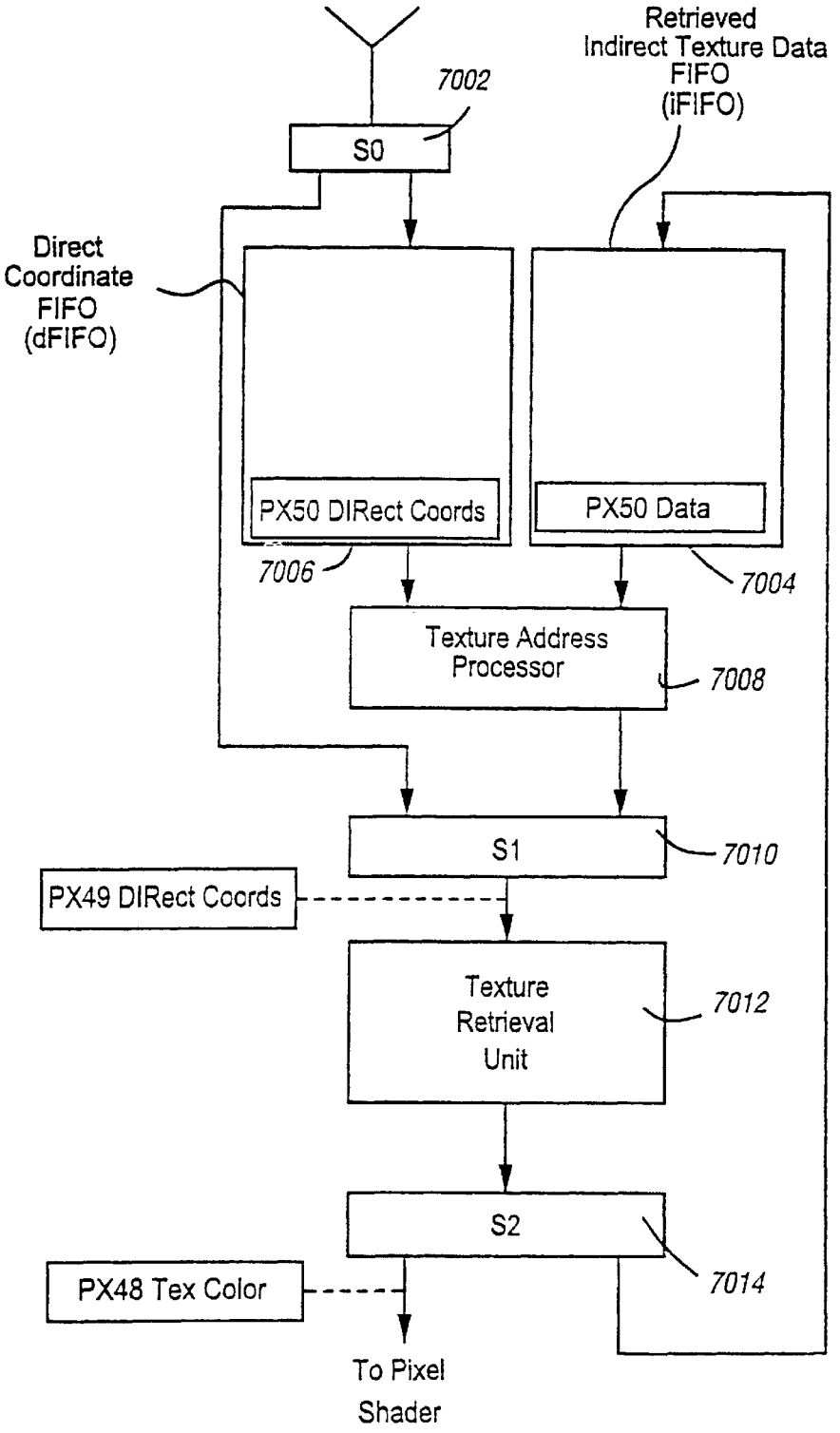

Retrieved
Indirect Texture Data
FIFO
(iFIFO)
7002
S0
Direct
Coordinate
FIFO
(dFIFO)
PX50 DIRect Coords
PX50 Data
7006
7004
Texture Address
Processor
7008
S1
7010
PX49 DIRect Coords
Texture
Retrieval
Unit
7012
S2
7014
PX48 Tex Color
To Pixel
Shader

Fig. 10K

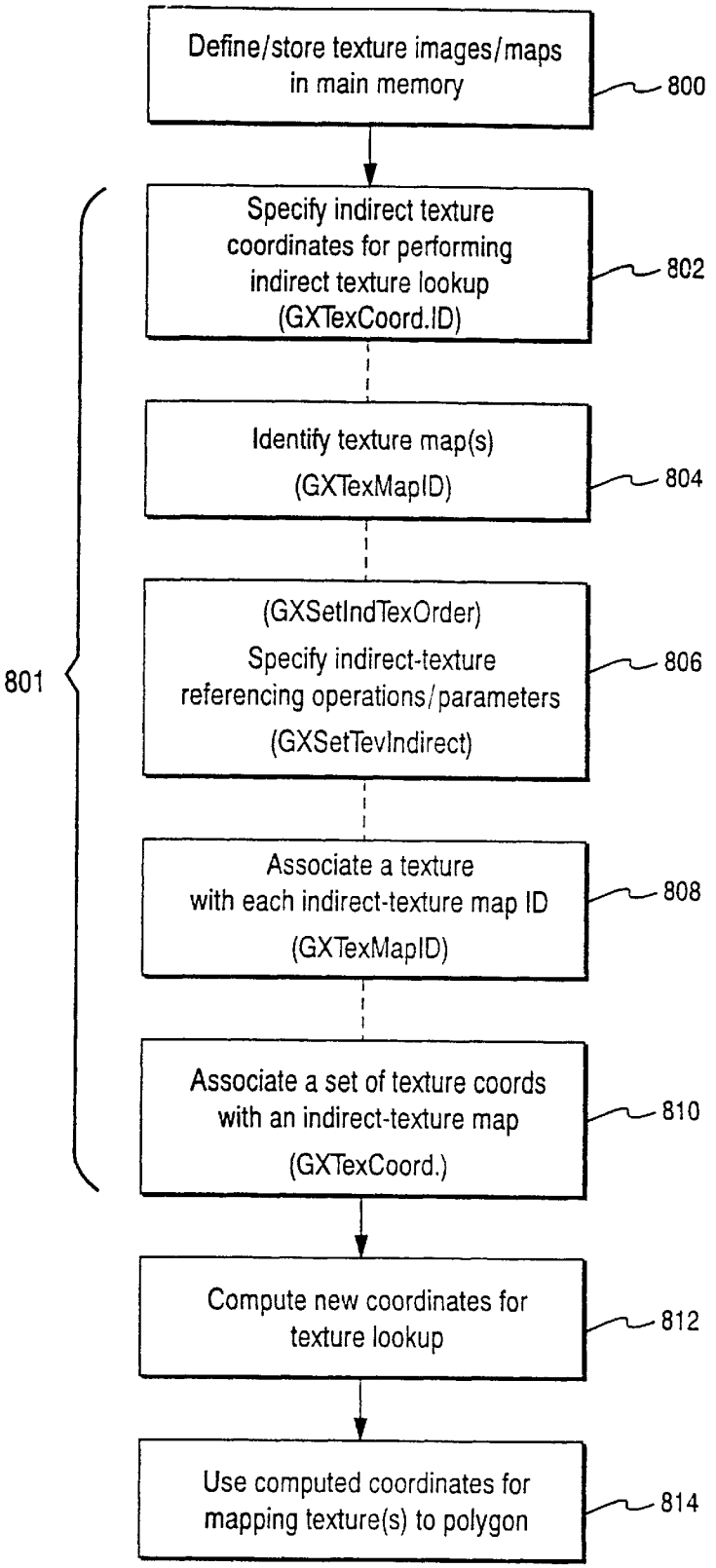

Define/store texture images/maps in main memory
800
Specify indirect texture coordinates for performing indirect texture lookup (GXTexCoord.ID)
802
Identify texture map(s) (GXTexMapID)
804
(GXSetIndTexOrder) Specify indirect-texture referencing operations/parameters (GXSetTevIndirect)
806
801
Associate a texture with each indirect-texture map ID (GXTexMapID)
808
Associate a set of texture coords with an indirect-texture map (GXTexCoord.)
810
Compute new coordinates for texture lookup
812
Use computed coordinates for mapping texture(s) to polygon
814

Fig. 11

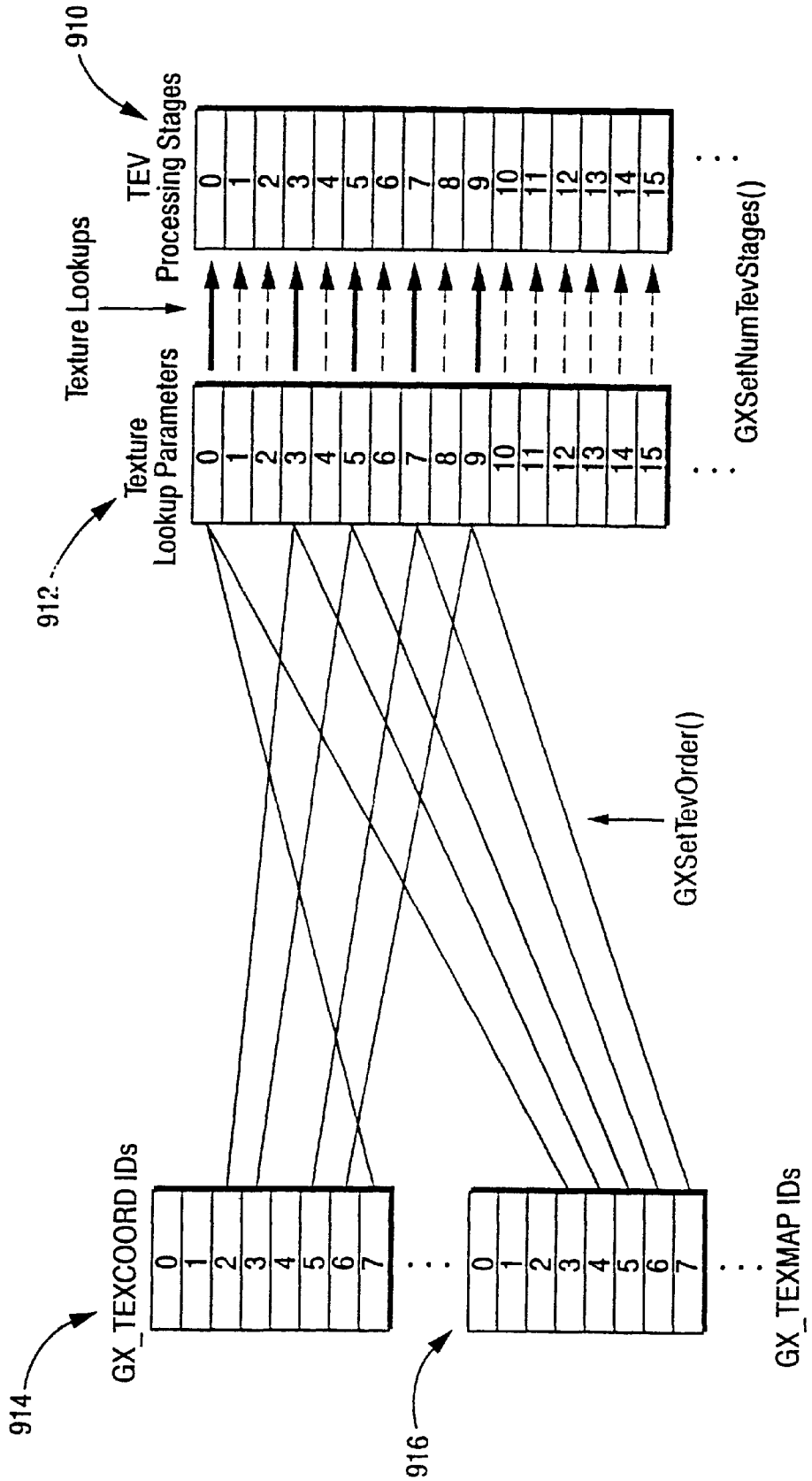


Fig. 12

REGULAR(NON-INDIRECT) TEXTURE PROCESSING

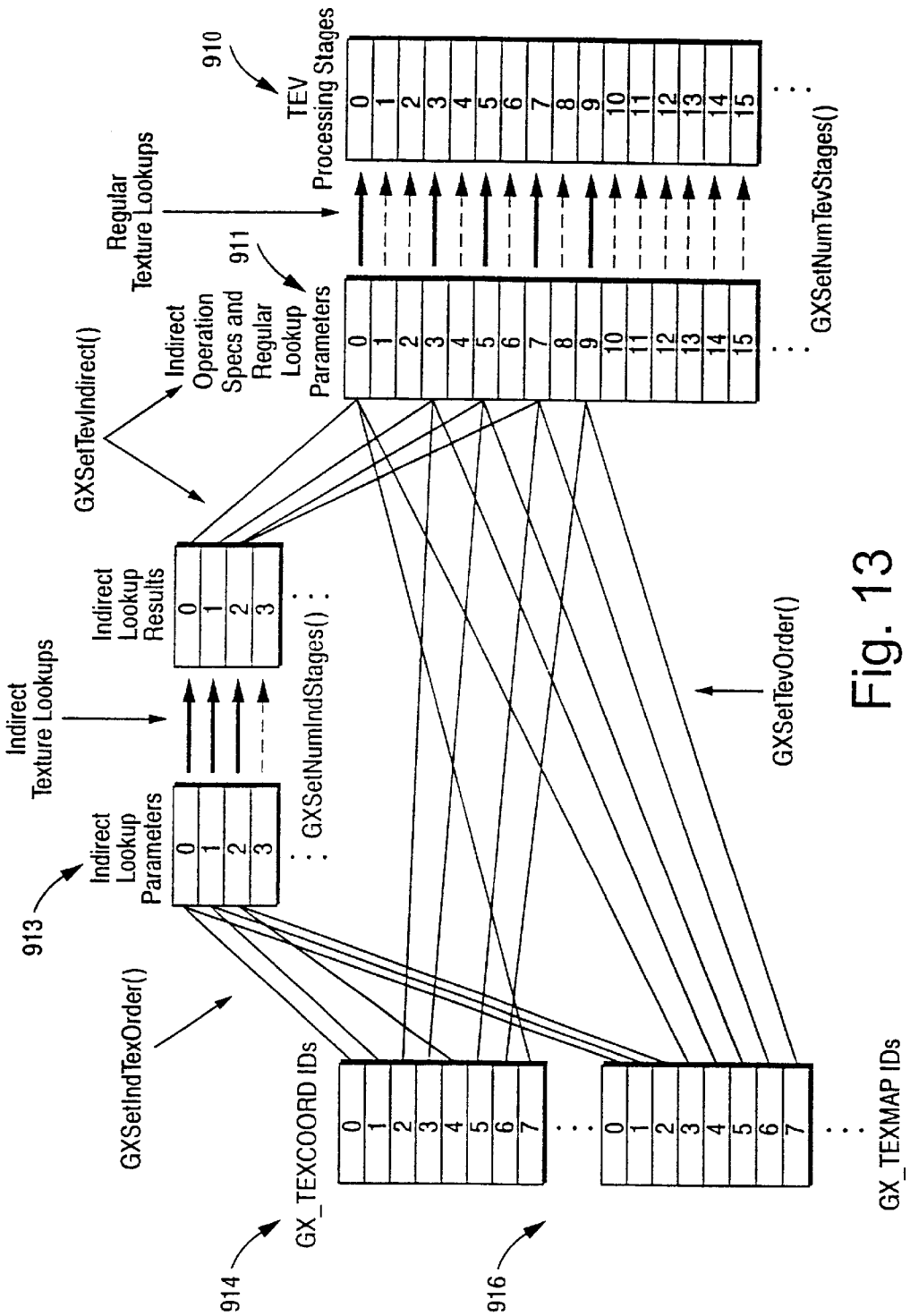


Fig. 13

REGULAR AND INDIRECT TEXTURE PROCESSING

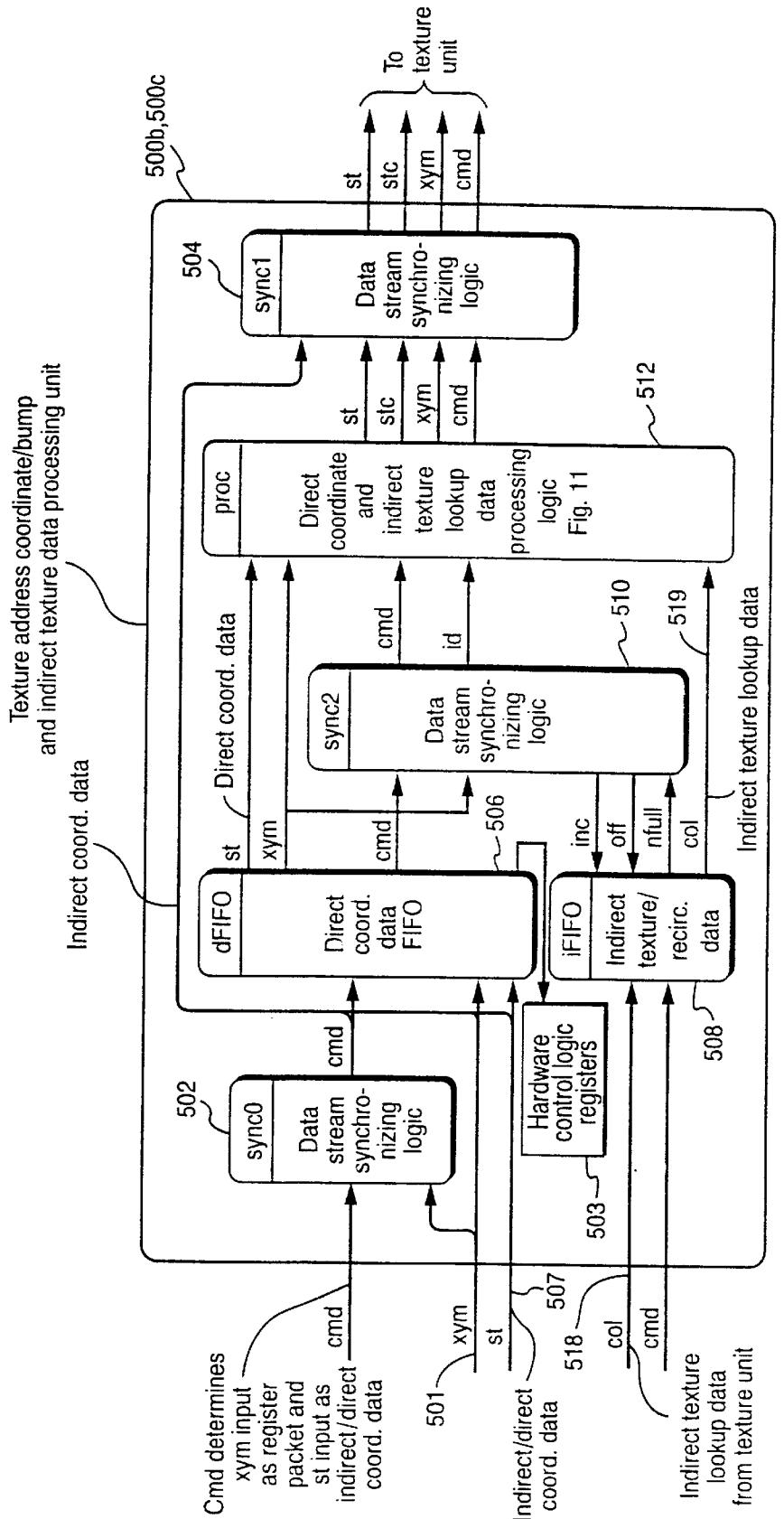


Fig. 14

EXAMPLE BUMP/TEXTURE COORDINATE PROCESSING UNIT

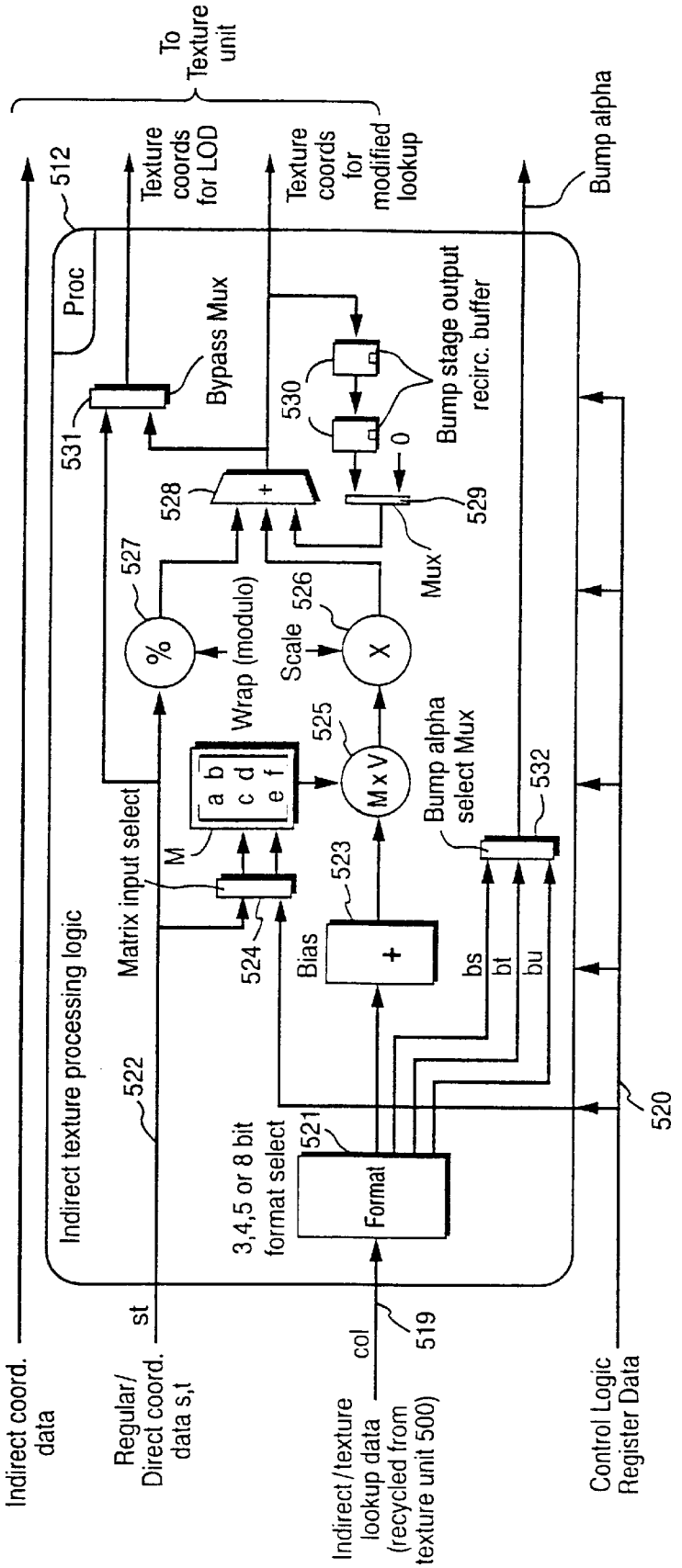


Fig. 15

EXAMPLE INDIRECT-TEXTURE LOOKUP DATA PROCESSING LOGIC

$$\begin{pmatrix} s' \\ t' \end{pmatrix} = \overbrace{\begin{pmatrix} ma & mb \\ mc & md \\ me & mf \end{pmatrix}}^{M} \cdot \overbrace{\begin{pmatrix} s \\ t \\ u \end{pmatrix}}^{V}$$

Fig. 16A

EXAMPLE TEXTURE OFFSET MATRICES

Matrix A

$$\begin{pmatrix} s/256 & t/256 \\ 0 & 0 \\ 0 & 0 \end{pmatrix}$$

Matrix B

$$\begin{pmatrix} 0 & 0 \\ s/256 & t/256 \\ 0 & 0 \end{pmatrix}$$

Fig. 16B

EXAMPLE TEXTURE OFFSET MATRICES

MTXA$_i$	s_i (1:0)	mb_i (10:0)				ma_i (10:0)			
MTXB$_i$	s_i (3:2)	md_i (10:0)				mc_i (10:0)			
MTXC$_i$	s_i (5:4)	mf_i (10:0)				me_i (10:0)			
CMDi		fb_i		tw_i	sw_i	m_i	$bias_i$	fmt_i	bt_i
	⋮								
							imask (7:0)		
GEN MODE		nbmp			ntev		ntex		

Fig. 17

EXAMPLE CONTROL LOGIC REGISTERS

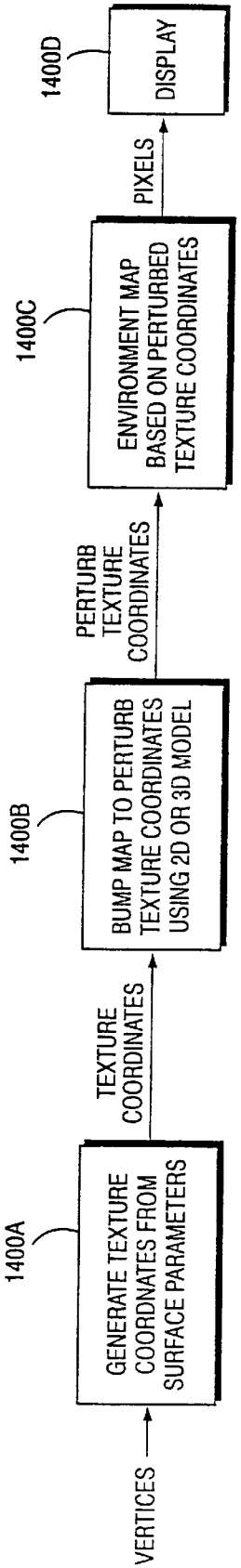


FIG. 18
EXAMPLE GENERAL ENVIRONMENT MAPPED BUMP MAPPING PROCEDURE

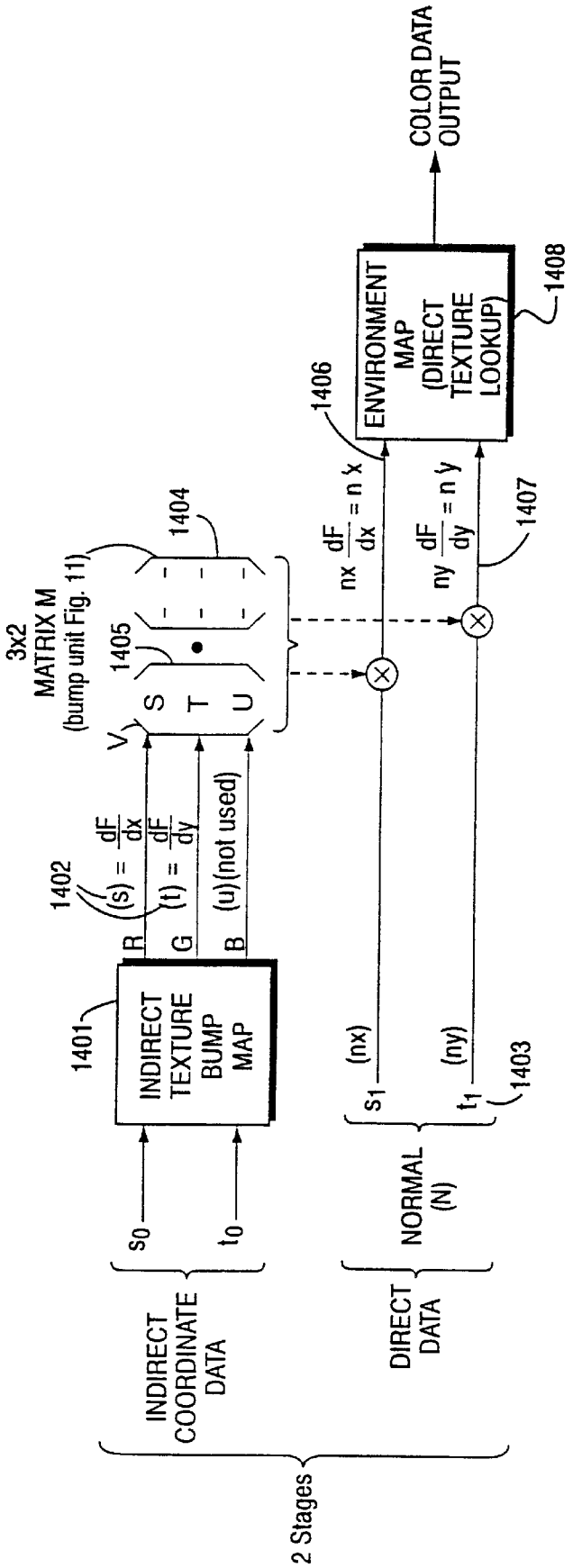


FIG. 19

METHOD 1: ENVIRONMENT-MAPPED BUMP MAPPING (EMBM) - BASED ON MODELED PERTURBATIONS(dXdYdZ)

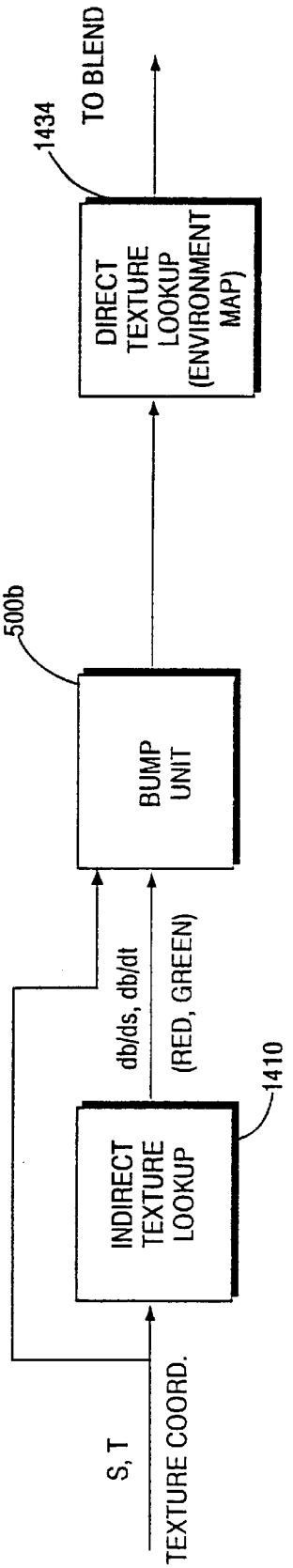


FIG. 20
EXAMPLE SIMPLIFIED FLAT SURFACE ENVIRONMENT MAPPED BUMP MAPPING

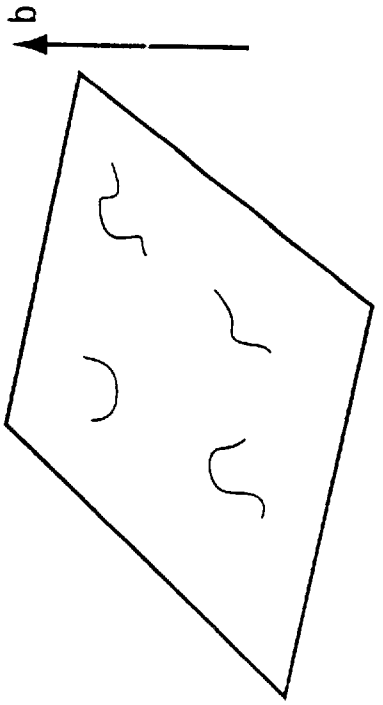


FIG. 22
EXAMPLE BUMP MAP

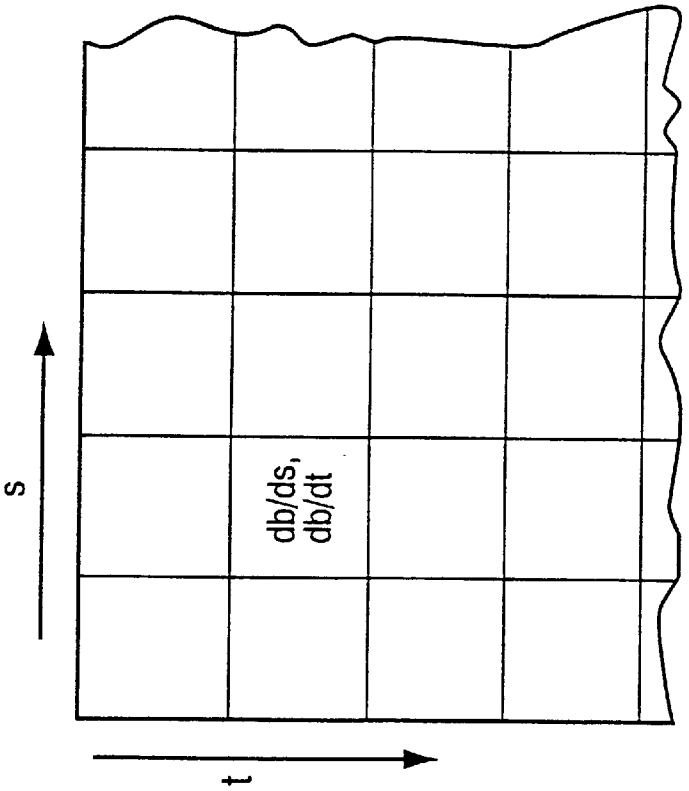


FIG. 21
EXAMPLE FLAT SURFACE
BUMP MAP TEXTURE

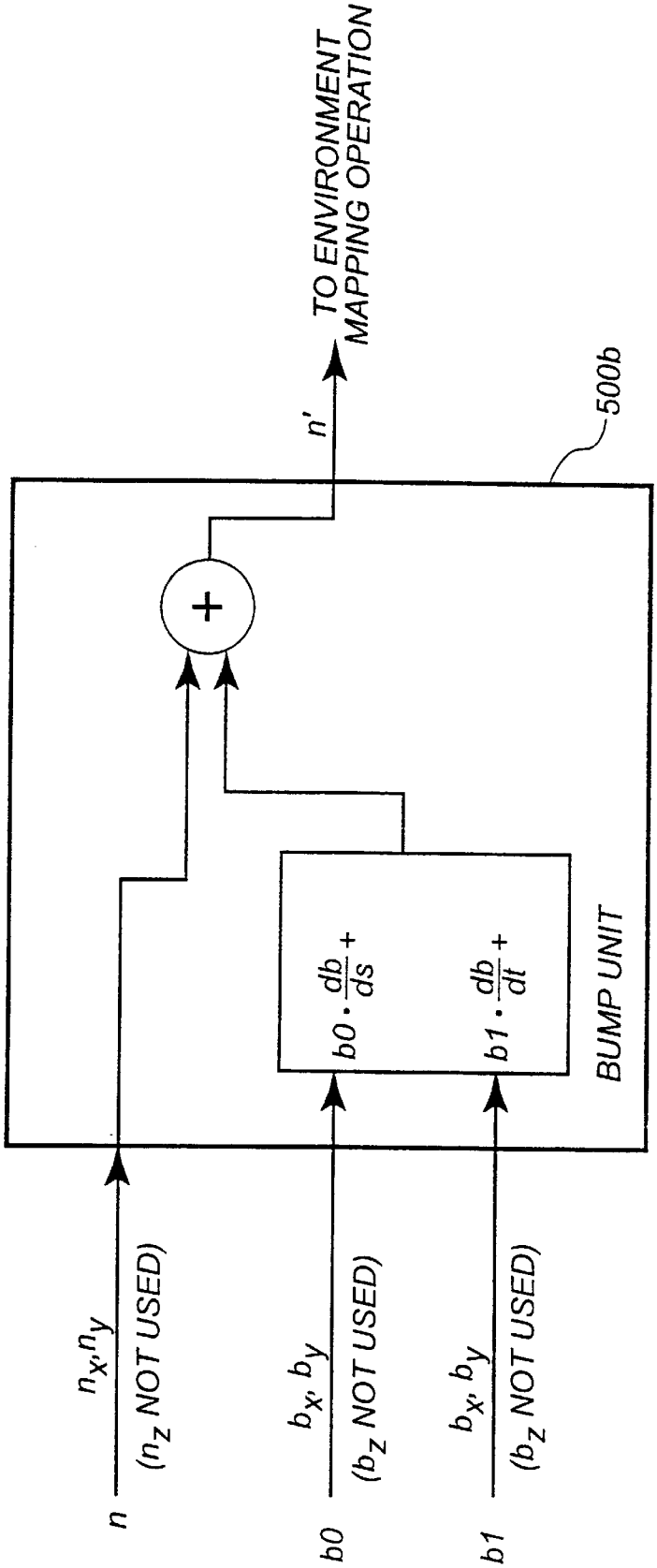


FIG. 23

EXAMPLE BUMP MAPPING FOR FLAT SURFACE EXAMPLE (dSdT)

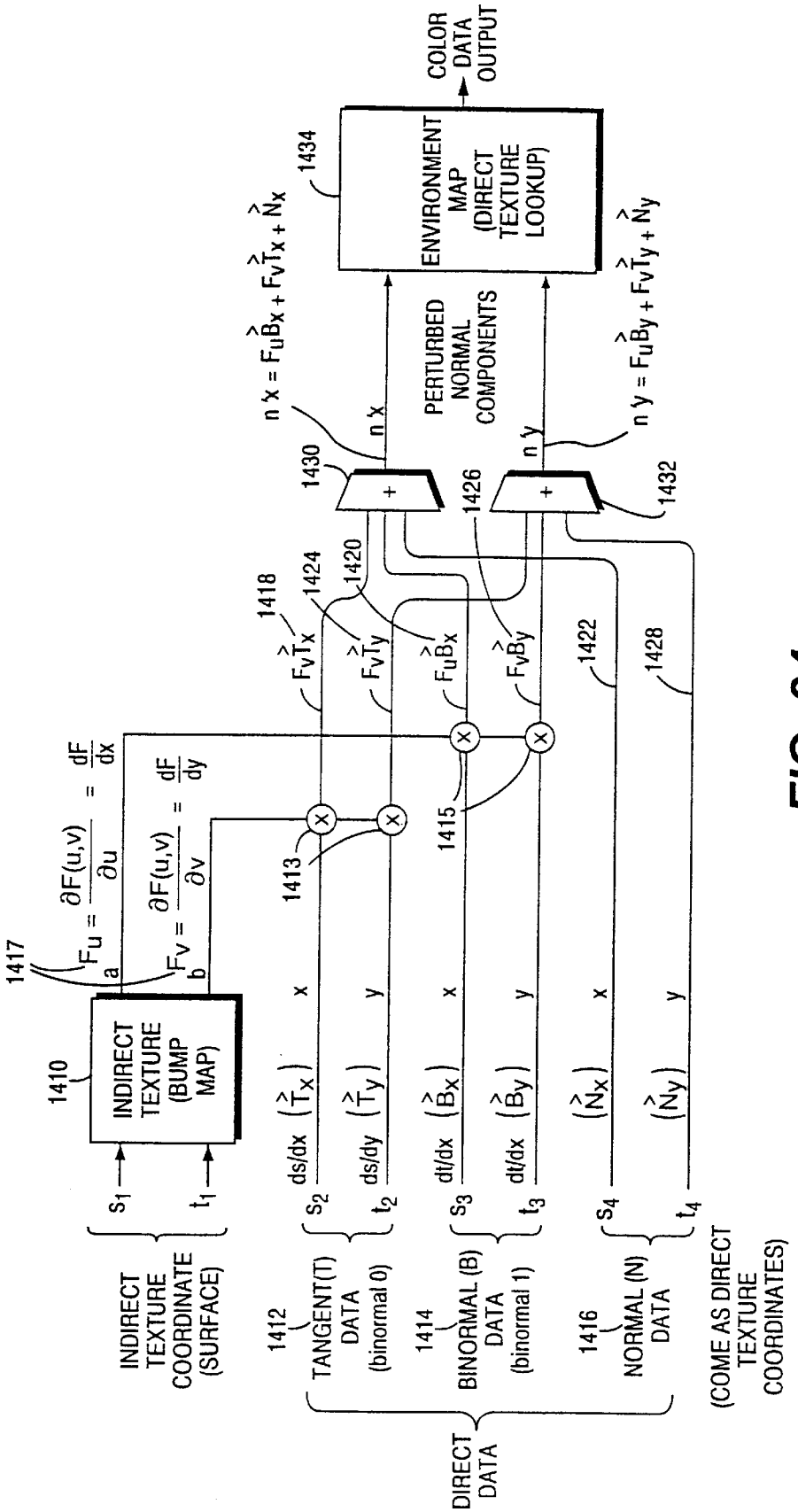


FIG. 24

METHOD 2: ENVIRONMENT - MAPPED BUMP MAPPING (EMBM) - BASED ON BINORMALS AND MODELED PERTURBATIONS (dSdT)

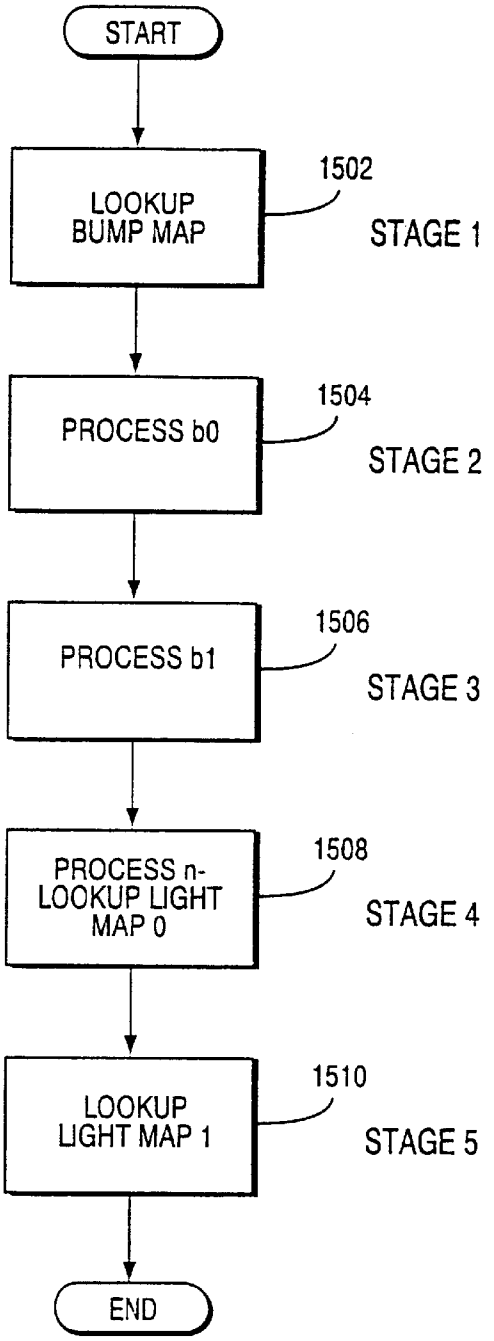


FIG. 25
EXAMPLE PROCESS FOR EMBM USING FLAT SURFACE (dSdT) MODELED PERTURBATIONS

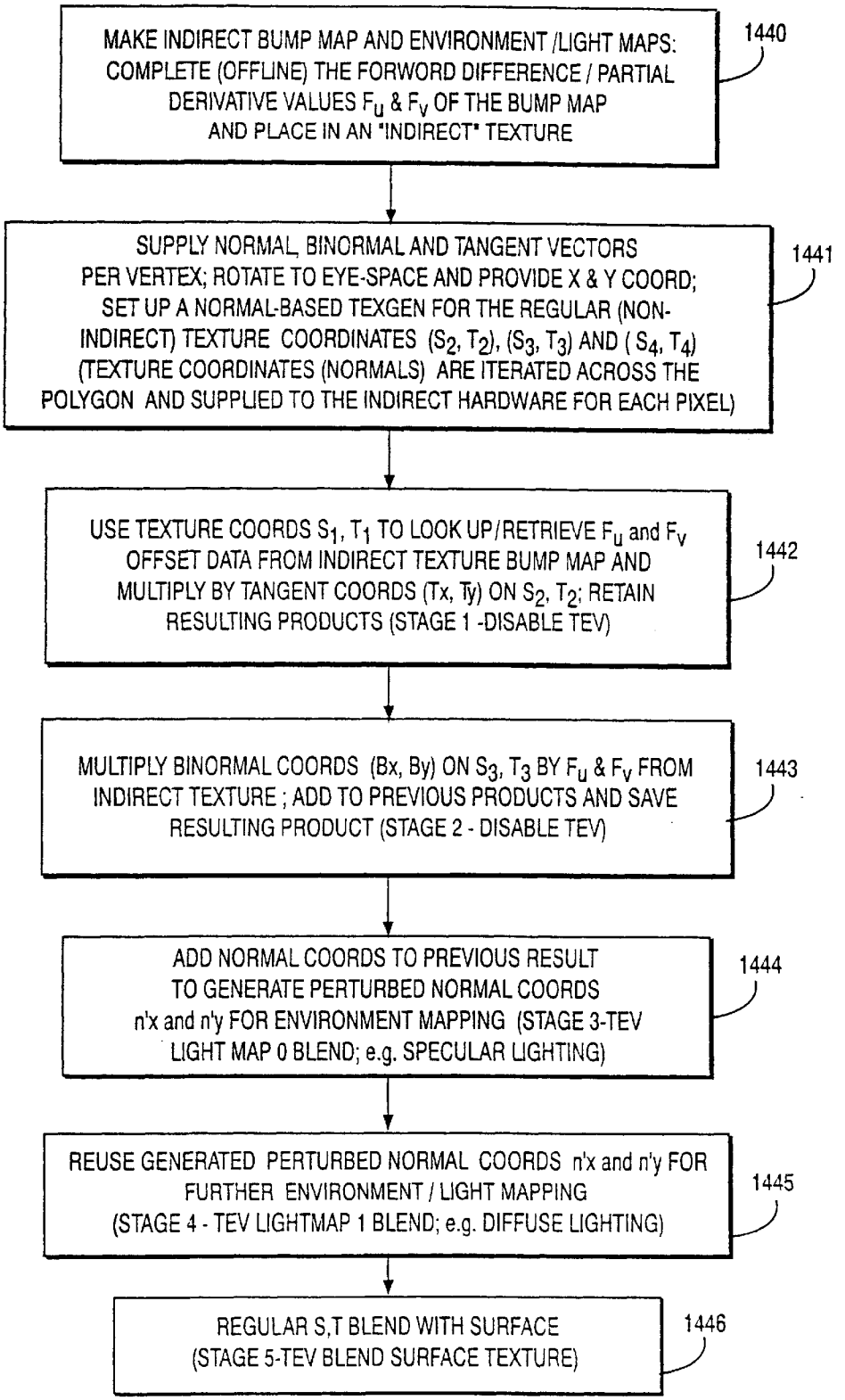


FIG. 26

EXAMPLE EMBM (dSdT) PROCESS STEPS

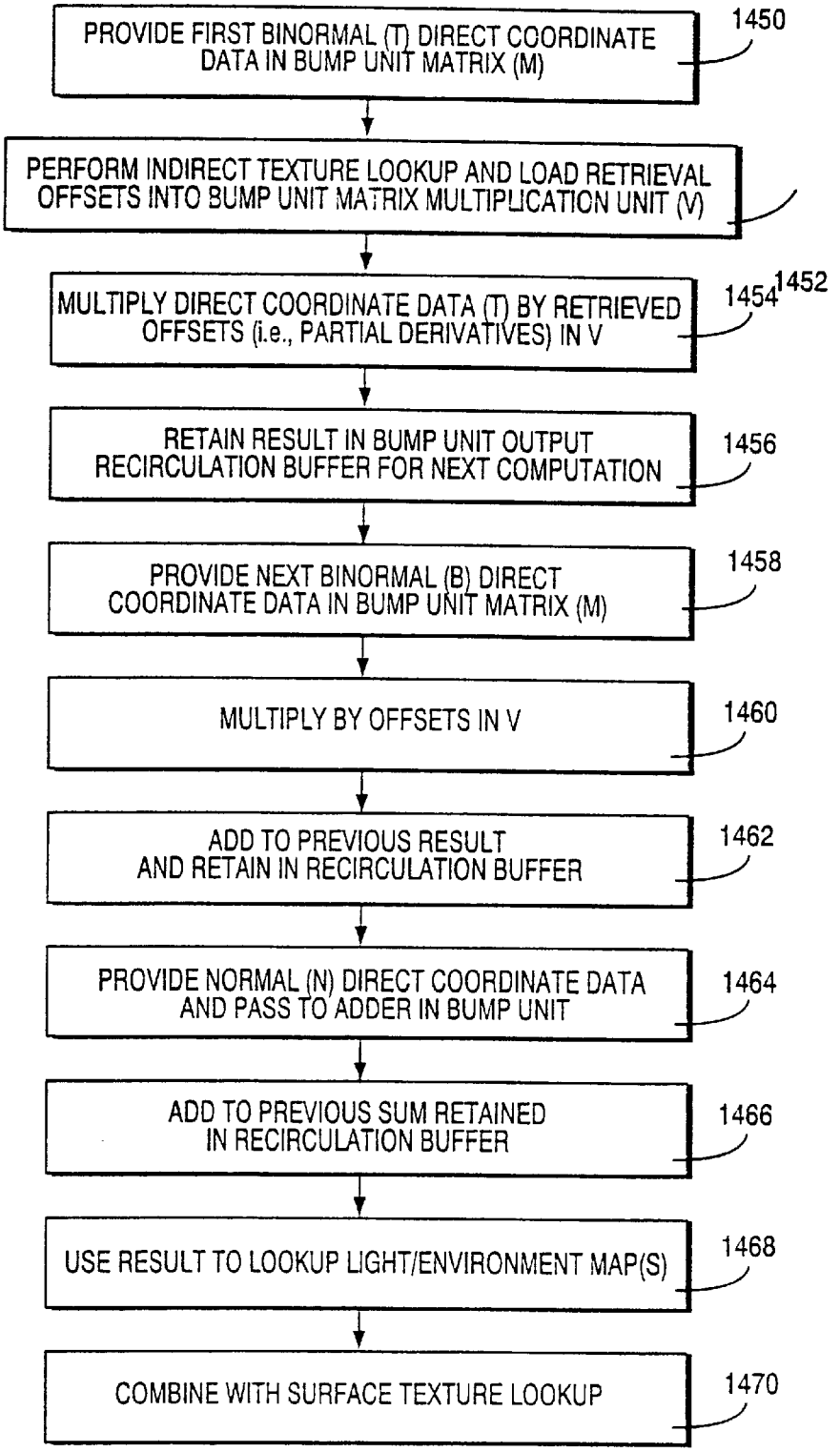


FIG. 27

DETAILED EXAMPLE EMBM (dSdT) PROCESS STEPS

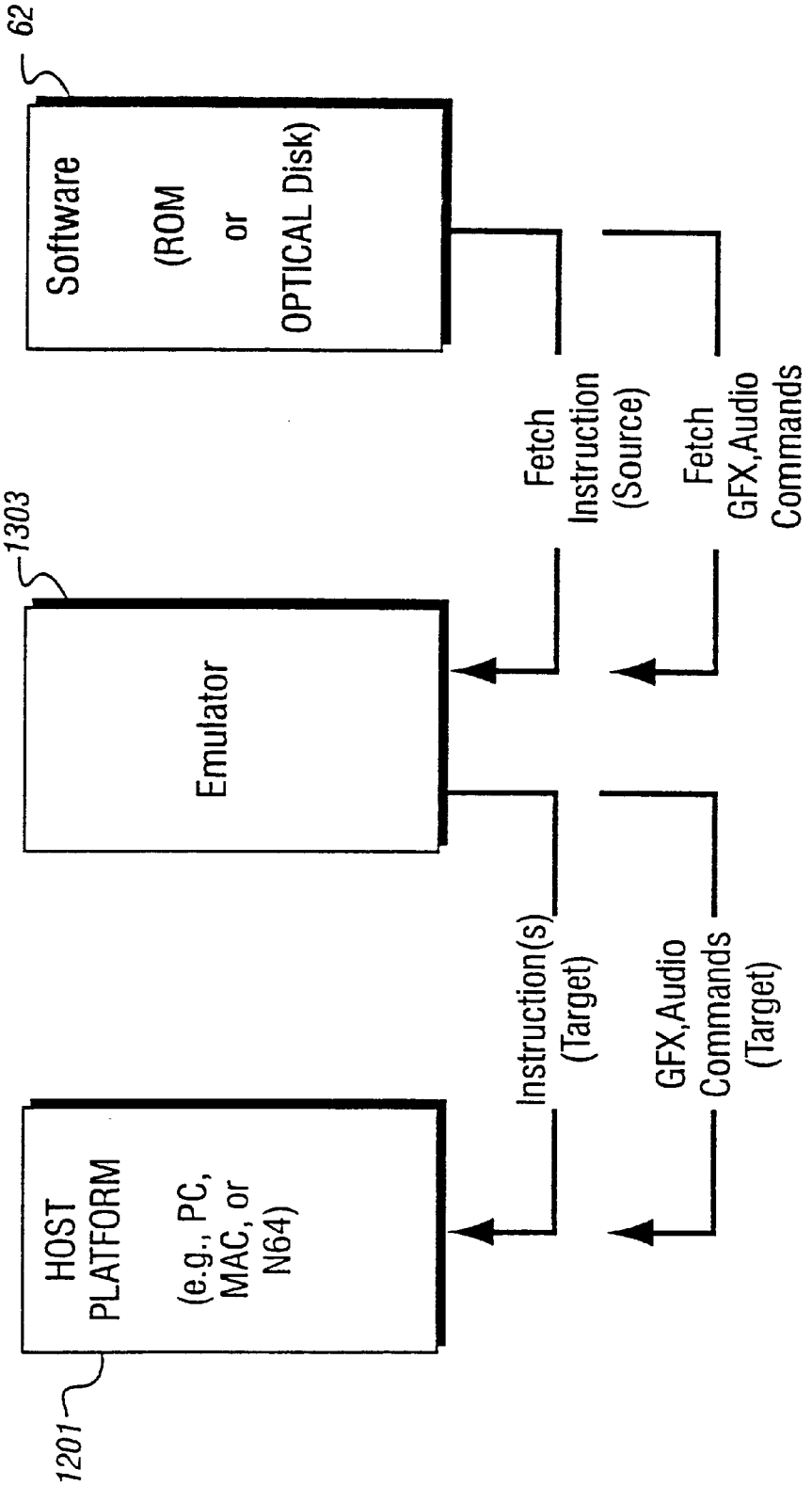

62
Software
(ROM
or
OPTICAL Disk)
1303
Emulator
1201
HOST
PLATFORM
(e.g., PC,
MAC, or
N64)
Fetch
Instruction
(Source)
Fetch
GFX,Audio
Commands
(Source)
Instruction(s)
(Target)
GFX,Audio
Commands
(Target)

Fig. 28

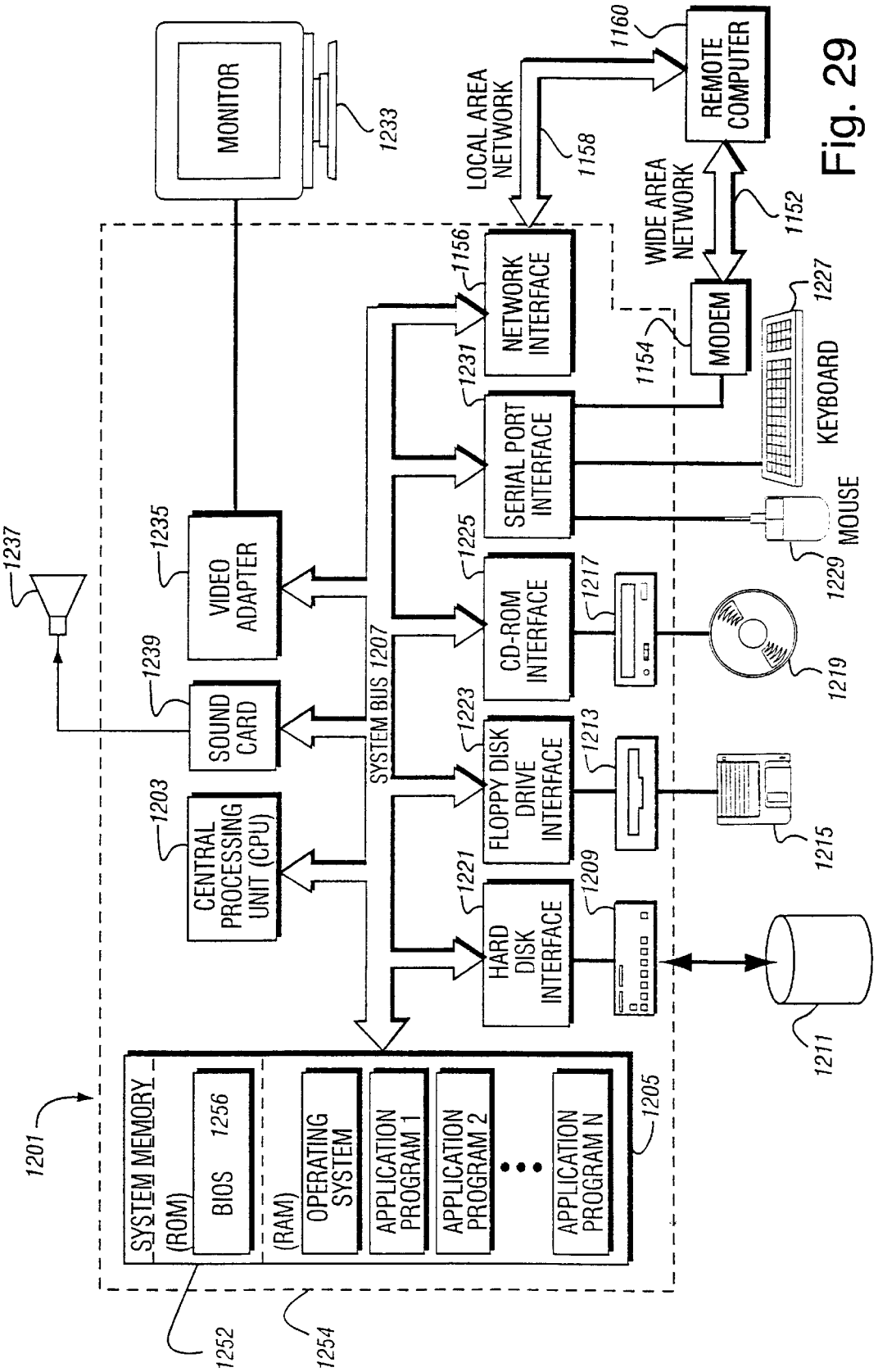

MONITOR
1233
LOCAL AREA NETWORK
1158
1160
REMOTE COMPUTER
WIDE AREA NETWORK
1152
1156
NETWORK INTERFACE
1231
SERIAL PORT INTERFACE
1154
MODEM
1227
KEYBOARD
MOUSE
1229
1237
1235
VIDEO ADAPTER
1225
CD-ROM INTERFACE
1217
1219
1239
SOUND CARD
SYSTEM BUS 1207
1223
FLOPPY DISK DRIVE INTERFACE
1213
1215
1203
CENTRAL PROCESSING UNIT (CPU)
1221
HARD DISK INTERFACE
1209
1211
1201
SYSTEM MEMORY
(ROM)
BIOS 1256
(RAM)
OPERATING SYSTEM
APPLICATION PROGRAM 1
APPLICATION PROGRAM 2
APPLICATION PROGRAM N
1205
1252
1254

Fig. 29

METHOD AND APPARATUS FOR ENVIRONMENT-MAPPED BUMP-MAPPING IN A GRAPHICS SYSTEM

RELATED APPLICATIONS

This application claims the benefit of U.S. Provisional Application Ser. No. 60/226,893, filed Aug. 23, 2000, the entire content of which is hereby incorporated by reference.

FIELD OF THE INVENTION

The present invention relates to computer graphics, and more particularly to interactive graphics systems such as home video game platforms. Still more particularly this invention relates to a method and apparatus for performing environment mapped bump-mapping in a graphics system.

BACKGROUND AND SUMMARY OF THE INVENTION

Many of us have seen films containing remarkably realistic dinosaurs, aliens, animated toys and other fanciful creatures. Such animations are made possible by computer graphics. Using such techniques, a computer graphics artist can specify how each object should look and how it should change in appearance over time, and a computer then models the objects and displays them on a display such as your television or a computer screen. The computer takes care of performing the many tasks required to make sure that each part of the displayed image is colored and shaped just right based on the position and orientation of each object in a scene, the direction in which light seems to strike each object, the surface texture of each object, and other factors.

Because computer graphics generation is complex, computer-generated three-dimensional graphics just a few years ago were mostly limited to expensive specialized flight simulators, high-end graphics workstations and supercomputers. The public saw some of the images generated by these computer systems in movies and expensive television advertisements, but most of us couldn't actually interact with the computers doing the graphics generation. All this has changed with the availability of relatively inexpensive 3D graphics platforms such as, for example, the Nintendo 64® and various 3D graphics cards now available for personal computers. It is now possible to interact with exciting 3D animations and simulations on relatively inexpensive computer graphics systems in your home or office.

One problem graphics system designers confronted in the past was how to create realistic looking surface detail on a rendered object without resorting to explicit modeling of the desired details with polygons or other geometric primitives. Although surface details can be simulated, for example, using myriad small triangles with interpolated shading between vertices, as the desired detail becomes finer and more intricate, explicit modeling with triangles or other primitives places high demands on the graphics system and becomes less practical. An alternative technique pioneered by E. Catmull and refined by J. F. Blinn and M. E. Newell is to "map" an image, either digitized or synthesized, onto a surface. (See "A Subdivision Algorithm for Computer Display of Curved Surfaces" by E. Catmull, Ph.D. Thesis, Report UTEC-CSc-74-133, Computer Science Department, University of Utah, Salt Lake City, Utah, December 1994 and "Texture and Reflection in Computer Generated Images" by J. F. Blinn and M. E. Newell, CACM, 19(10), October 1976, 452–457). This approach is known as texture mapping (or pattern mapping) and the image is called a

texture map (or simply referred to as a texture). Alternatively, the texture map may be defined by a procedure rather than an image.

Typically, the texture map is defined within a 2D rectangular coordinate space and parameterized using a pair of orthogonal texture coordinates such, as for example, (u, v) or (s, t). Individual elements within the texture map are often called texels. At each rendered pixel, selected texels are used either to substitute for or to scale one or more material properties of the rendered object surface. This process is often referred to as texture mapping or "texturing."

Although most 3-D graphics rendering systems now include a texturing subsystem for retrieving textures from memory and mapping the textures onto a rendered object surface, another problem confronting graphics system designers is how to texture a 3D object with realistic-looking surface characteristics that react to various lighting conditions in a manner similar to the surface of an actual object having, for example, random surface flaws, irregularities, roughness, bumps or other slight non-planar surface variations. Regular texture mapping does not provide such realism because texture images are generally two-dimensional and can not react or change in appearance in response to changes in position or direction of the lighting within a rendered scene. While in some instances such minute surface characteristics might be actually modeled, the time required for translating and rendering a 3D object with such a complex surface would be prohibitive for most real-time or interactive gaming applications. Consequently, various solutions to this problem were offered. For example, a technique generally known as "bump-mapping" was developed which allowed one to approximate the effect that non-planar surface variations would produce on lighted object. See, for example, J. F. Blinn "Simulation of Wrinkled Surfaces" *Computer Graphics*, (SIGRAPH '78 Proceedings), vol. 12, No. 3, pp. 286–292 (August 1978) and "Models of Light Reflection for Computer Synthesized Pictures", Proc. 4th Conference on Computer Graphics and Instructive Techniques, 1977.

Basically, bump-mapping allows a graphics application programmer to add realism to the surface of an object without using a lot of geometry by modeling small surface variations as height differences and then applying those difference values over a surface as perturbations to a surface Normal vector used in computing surface lighting effects. Effectively, a bump-map modifies the shading of a polygon (or primitive) by perturbing the surface Normal on a per-pixel basis. The shading makes the surface appear bumpy, even though the underlying geometry is relatively flat.

Although conventional bump-mapping techniques offer some improvement, a further problem confronting graphics system designers is that conventional approaches to simple forms of bump-mapping, such as the "emboss"-style of bump mapping, typically only consider the interaction between a bump map and a single diffuse light source. Moreover, in many cases, the bump map is inconveniently "tied" to a specific geometric shape in the sense that it should be constructed (i.e., parameterized) for an object having a particular surface geometry and only works well for geometrically similar surfaces. In addition, if the object is rotated, the bump map will not apply to the entire surface (e.g., newly visible surface regions after rotation).

Another known technique for providing realistic looking surface on a rendered object uses a method called "environment mapping". Environment mapping (EM), also called "reflection mapping", is a simple yet powerful method of

generating approximations of reflections in curved surfaces. This technique was introduced by Blinn and Newell, "Texture and Reflection In Computer Generated Images," *Communications of the ACM*. Vol. 19, no. 10, pp. 542–547 (October 1976). Conventionally, methods start with a ray from the viewer to a point on the reflector. This ray is then reflected with respect to the normal at that point. Instead of finding the intersection with the closest surface, as is done in ray tracing processes, EM uses the direction of the reflection vector as an index to an image containing the environment. The environment mapping approximation assumes that the objects and lights being reflected with EM are far away, and that the reflector will not reflect itself. If these assumptions hold, then the environment around the reflector can be treated as a two-dimensional projection surrounding it.

The basic steps for environment mapping are as follows:

Generate a two-dimensional image of the environment (this is the environmental map).

For each pixel that contains a reflective object, compute the normal at the location on the surface of the object (if per-pixel EM is not available, then the normal is computed at polygon vertices).

Compute the reflection vector from the view vector and the normal.

Use the reflection vector to compute an index into the environment map that represents the objects in the reflection direction.

Use the data from the environment map to color the current pixel.

In addition, there are a variety of known "projector" functions that may be used to map the reflection vector into one or more textures. Blinn and Newell's algorithm and Grene's cubic environment mapping technique are classic mapping methods, although a sphere map is also commonly used.

While much work has been done in the past, further improvements are possible and desirable.

The present invention improves on prior techniques by providing an apparatus and example procedures for performing an environment-mapped bump-mapping (EMBM). In accordance with one aspect provided by this invention, environment mapped bump mapping in a graphics system is performed by generating texture coordinates from parameters of a surface; bump mapping to perturb the texture coordinates; environment mapping based on the perturbed texture coordinates; and displaying an image obtained from the environment mapping. The bump mapping may model surface perturbations using a three-dimensional model in modeling space, or it may model the surface perturbations in two dimensions in eye-space. These different modeling techniques provide different advantages and features.

In more detail, one environment mapped bump mapping technique uses a bump map to model surface perturbations in three dimensions. The output of the bump mapping operation is provided to a matrix multiplication operation that allows rotation in any direction. The resulting rotated values are combined with Normal vector values for environment mapping.

In another embodiment, the bump map models surface perturbations in two dimensions using partial derivatives. The bump map output is combined with Normal vector partial derivatives using a sum-of-product calculation to provide texture coordinates for environment mapping. The inverse of the transformation matrix is used for geometric transformations to transform the Normal vector partial derivatives.

In more detail, a first example EMBM procedure is provided that permits very fast processing of object surface Normal perturbations modeled in 3D (dXdYdZ) for specific object surface geometries using a pre-computed bump map stored in memory as a texture. A second example EMBM procedure is provided for Normal perturbations modeled with respect to a flat surface (dSdT) which frees the bump map from use with a particular surface geometry and allows it to be applied with environment mapping/lighting to a variety of different rendered object shapes.

The EMBM method provided by an aspect of the present invention uses a pre-constructed bump-map texture accessed as an indirect texture along with pre-computed computed object surface normals (i.e., the Normal, Tangent and Binormal vectors) from each vertex of rendered polygons to effectively generate a new perturbed Normal vector per vertex. The new perturbed Normal vectors are then used to look up an environment texture map. For example, the environment texture map may be a spherical reflection map of an environment surrounding a rendered object or a texture map containing lighting or other data for creating more complex surface lighting effects.

The example embodiment provides a specialized texture data/coordinate processing "bump unit" in the graphics pipeline for performing predetermined matrix multiplication operations (e.g., modeling-space to eye-space conversions) on retrieved lookup data from the indirect-texture bump map. The bump unit (e.g., see FIG. **15**) uses a programmable 3×2 element multiplication matrix (M) and scaling circuits that also permits rotation of bump-mapped objects about multiple orthogonal axes. The bump unit is also provided with an output storage buffer (**530**) for reusing computed coordinate data output values in subsequent processing cycles.

In accordance with a further aspect provided by the invention, object surface Normal perturbations for one or more specific surface geometries are modeled in 3D using a pre-computed dXdYdZ bump map stored in memory as a texture prior to running a particular graphics application (i.e., texel values within an individual bump-map specify surface Normal vector offsets per vertex in 3D model space). A texturing pipeline implemented in hardware then performs an environment-mapped bump-mapped dXdYdZ texture lookup. In the disclosed example embodiment, this is implemented using an indirect texture look-up to obtain the dXdYdZ bump map data to compute a perturbed texture coordinate for lookup into, for example, a spherical reflection environment map. At runtime, the graphics application matches an object having a particular surface geometry with the pre-computed dXdYdZ bump-map texture for that geometry and accesses the bump-map as an "indirect" texture.

A programmable 3×2 multiplication matrix can be implemented by bump unit hardware to provide improved versatility when rendering and displaying such dXdYdZ bump-mapped objects. For example, the matrix may be programmably loaded with appropriate transform constants for allowing conversion of texture coordinate components to eye-space or dynamically loaded with texture coordinates for implementing dXdYdZ bump-mapped texturing while rotating an object about two or more orthogonal axes. The dXdYdZ bump-map indirect look-up data is then used along with per vertex Normal-generated texture coordinate components to compute the perturbed texture coordinates for the look-up into an environment map.

Although the dXdYdZ bump-map textures are custom parameterized for individual surface geometries and,

consequently, are effectively "tied" to the particular object for which they were designed, this approach to EMBM is advantageous in that processing is very fast, requiring only a single texture coordinate processing stage corresponding to a single shading/color-blending stage. Moreover, in a preferred implementation of the present invention, the bump-mapped objects may be dynamically rotated about at least two orthogonal axes without disruption to the bump-mapped effect on the object surface.

In accordance with another aspect provided by the invention, object surface Normal perturbations are initially modeled with respect to a flat surface, then during application runtime, the Normal perturbations may be mapped to a rendered object surface having a different or arbitrary geometric configuration. A pre-computed bump dSdT map containing partial derivative data is stored in memory as a 2D texture and subsequently accessed as an "indirect" texture. The pre-computed dSdT bump map data may be derived, for example, by forward differencing the data from a conventional paint program 2D or other image. Basically, Normal, Tangent and Binormal vectors, which are specified per vertex as polygon vertex attributes of a modeled 3D object surface, are first rotated (transformed) into view-space (eye-space/camera-space) and then used along with the precomputed bump map data to compute a perturbed Normal for each vertex. The perturbed Normal is then used to look up a light or environment map. In other words, Binormal- and Tangent-generated texture coordinate components are scaled (multiplied) by the pre-computed partial derivative dSdT bump map data (stored as an indirect texture) and then are combined with Normal vector generated texture coordinate components over several cycles/stages. The resulting computed texture coordinates correspond to an appropriately perturbed Normal vector, which is then used to look-up a light or environment map.

BRIEF DESCRIPTION OF THE DRAWINGS

These and other features and advantages provided by the invention will be better and more completely understood by referring to the following detailed description of presently preferred embodiments in conjunction with the drawings, of which:

FIG. **1** is an overall view of an example interactive computer graphics system;

FIG. **2** is a block diagram of the FIG. **1** example computer graphics system;

FIG. **3** is a block diagram of the example graphics and audio processor shown in FIG. **2**;

FIG. **4** is a block diagram of the example 3D graphics processor shown in FIG. **3**;

FIG. **5** is an example logical flow diagram of the FIG. **4** graphics processor;

FIG. **6** is block diagram illustrating a logical overview of indirect texture processing in accordance with the present invention;

FIG. **7**A is a functional block diagram illustrating a simple basic example of a regular (non-indirect) texture look-up;

FIG. **7**B is a functional block diagram illustrating a simple basic example of an indirect texture lookup in accordance with the present invention;

FIG. **8** is a block diagram illustrating an overview of an example physical configuration for indirect texture processing in accordance with the present invention;

FIG. **9** is a block diagram illustrating a logical overview of the texture address (coordinate/data) processor operation;

FIGS. **10**A–**10**K are a series of block diagrams illustrating the relative progression of pixel direct coordinate data and pixel indirect texture data in the example texturing pipeline implementation as a result of interleaved direct and indirect texture processing;

FIG. **11** is a flow chart illustrating example steps for implementing indirect texture processing in accordance with the present invention;

FIG. **12** is a functional operations diagram illustrating an example of regular (non-indirect) texture processing in accordance with the present invention;

FIG. **13** is a functional operations diagram illustrating an example of both regular (non-indirect) and indirect texture processing in accordance with the present invention;

FIG. **14** is a block diagram showing a detailed example of the texture coordinate/bump processing unit shown in FIG. **5**;

FIG. **15** is a block diagram showing a detailed example of the indirect texture lookup data/coordinate processing logic (proc) shown in FIG. **14**;

FIGS. **16**A and **16**B show example texture offset matrices used by processing logic circuit (proc) of FIG. **15**;

FIG. **17** is a block diagram illustrating example data field formats of control logic registers for controlling the operations within the processing circuitry of FIG. **15**;

FIG. **18** shows an example environment mapped bump mapping process;

FIG. **19** is a logical block diagram illustrating an example process of environment-mapped bump mapping using Normal vector perturbations modeled in 3D (dXdYdZ);

FIG. **20** is a simplified flat surface environment mapped bump mapping;

FIG. **21** shows an example flat surface bump map texture;

FIG. **22** shows an example flat surface bump map;

FIG. **23** a logical block diagram illustrating a simplified example of bump mapping for the flat surface (dSdT) embodiment;

FIG. **24** is a logical block diagram illustrating an example process of environment-mapped bump mapping using Normal vector perturbations modeled with respect to a flat surface (dSdT);

FIG. **25** shows an example process for environment mapped bump mapping modeled using the flat surface (dSdT) approach;

FIG. **26** is a flowchart of example steps performed for implementing the FIG. **24** embodiment;

FIG. **27** is a more detailed flowchart of example steps for implementing the FIG. **24** embodiment; and

FIGS. **28** and **29** show example alternative compatible implementations.

DETAILED DESCRIPTION OF EXAMPLE EMBODIMENTS OF THE INVENTION

FIG. **1** shows an example interactive 3D computer graphics system **50**. System **50** can be used to play interactive 3D video games with interesting stereo sound. It can also be used for a variety of other applications.

In this example, system **50** is capable of processing, interactively in real time, a digital representation or model of a three-dimensional world. System **50** can display some or all of the world from any arbitrary viewpoint. For example, system **50** can interactively change the viewpoint in response to real time inputs from handheld controllers **52***a*,

52*b* or other input devices. This allows the game player to see the world through the eyes of someone within or outside of the world. System **50** can be used for applications that do not require real time 3D interactive display (e.g., 2D display generation and/or non-interactive display), but the capability of displaying quality 3D images very quickly can be used to create very realistic and exciting game play or other graphical interactions.

To play a video game or other application using system **50**, the user first connects a main unit **54** to his or her color television set **56** or other display device by connecting a cable **58** between the two. Main unit **54** produces both video signals and audio signals for controlling color television set **56**. The video signals are what controls the images displayed on the television screen **59**, and the audio signals are played back as sound through television stereo loudspeakers **61**L, **61**R.

The user also needs to connect main unit **54** to a power source. This power source may be a conventional AC adapter (not shown) that plugs into a standard home electrical wall socket and converts the house current into a lower DC voltage signal suitable for powering the main unit **54**. Batteries could be used in other implementations.

The user may use hand controllers **52***a*, **52***b* to control main unit **54**. Controls **60** can be used, for example, to specify the direction (up or down, left or right, closer or further away) that a character displayed on television **56** should move within a 3D world. Controls **60** also provide input for other applications (e.g., menu selection, pointer/cursor control, etc.). Controllers **52** can take a variety of forms. In this example, controllers **52** shown each include controls **60** such as joysticks, push buttons and/or directional switches. Controllers **52** may be connected to main unit **54** by cables or wirelessly via electromagnetic (e.g., radio or infrared) waves.

To play an application such as a game, the user selects an appropriate storage medium **62** storing the video game or other application he or she wants to play, and inserts that storage medium into a slot **64** in main unit **54**. Storage medium **62** may, for example, be a specially encoded and/or encrypted optical and/or magnetic disk. The user may operate a power switch **66** to turn on main unit **54** and cause the main unit to begin running the video game or other application based on the software stored in the storage medium **62**. The user may operate controllers **52** to provide inputs to main unit **54**. For example, operating a control **60** may cause the game or other application to start. Moving other controls **60** can cause animated characters to move in different directions or change the user's point of view in a 3D world. Depending upon the particular software stored within the storage medium **62**, the various controls **60** on the controller **52** can perform different functions at different times.

Example Electronics of Overall System

FIG. **2** shows a block diagram of example components of system **50**. The primary components include:

- a main processor (CPU) **110**,
- a main memory **112**, and
- a graphics and audio processor **114**.

In this example, main processor **110** (e.g., an enhanced IBM Power PC 750) receives inputs from handheld controllers **108** (and/or other input devices) via graphics and audio processor **114**. Main processor **110** interactively responds to user inputs, and executes a video game or other program supplied, for example, by external storage media **62** via a mass storage access device **106** such as an optical disk drive. As one example, in the context of video game play, main processor **110** can perform collision detection and animation processing in addition to a variety of interactive and control functions.

In this example, main processor **110** generates 3D graphics and audio commands and sends them to graphics and audio processor **114**. The graphics and audio processor **114** processes these commands to generate interesting visual images on display **59** and interesting stereo sound on stereo loudspeakers **61**R, **61**L or other suitable sound-generating devices.

Example system **50** includes a video encoder **120** that receives image signals from graphics and audio processor **114** and converts the image signals into analog and/or digital video signals suitable for display on a standard display device such as a computer monitor or home color television set **56**. System **50** also includes an audio codec (compressor/decompressor) **122** that compresses and decompresses digitized audio signals and may also convert between digital and analog audio signaling formats as needed. Audio codec **122** can receive audio inputs via a buffer **124** and provide them to graphics and audio processor **114** for processing (e.g., mixing with other audio signals the processor generates and/or receives via a streaming audio output of mass storage access device **106**). Graphics and audio processor **114** in this example can store audio related information in an audio memory **126** that is available for audio tasks. Graphics and audio processor **114** provides the resulting audio output signals to audio codec **122** for decompression r and conversion to analog signals (e.g., via buffer amplifiers **128**L, **128**R) so they can be reproduced by loudspeakers **61**L, **61**R.

Graphics and audio processor **114** has the ability to communicate with various additional devices that may be present within system **50**. For example, a parallel digital bus **130** may be used to communicate with mass storage access device **106** and/or other components. A serial peripheral bus **132** may communicate with a variety of peripheral or other devices including, for example:

- a programmable read-only memory and/or real time clock **134**,
- a modem **136** or other networking interface (which may in turn connect system **50** to a telecommunications network **138** such as the Internet or other digital network from/to which program instructions and/or data can be downloaded or uploaded), and

flash memory **140**.

A further external serial bus **142** may be used to communicate with additional expansion memory **144** (e.g., a memory card) or other devices. Connectors may be used to connect various devices to busses **130**, **132**, **142**.

Example Graphics And Audio Processor

FIG. **3** is a block diagram of an example graphics and audio processor **114**. Graphics and audio processor **114** in one example may be a single-chip ASIC (application specific integrated circuit). In this example, graphics and audio processor **114** includes:

- a processor interface **150**,
- a memory interface/controller **152**,
- a 3D graphics processor **154**,
- an audio digital signal processor (DSP) **156**,
- an audio memory interface **158**,
- an audio interface and mixer **160**,
- a peripheral controller **162**, and
- a display controller **164**.

3D graphics processor **154** performs graphics processing tasks. Audio digital signal processor **156** performs audio

processing tasks. Display controller **164** accesses image information from main memory **112** and provides it to video encoder **120** for display on display device **56**. Audio interface and mixer **160** interfaces with audio codec **122**, and can also mix audio from different sources (e.g., streaming audio from mass storage access device **106**, the output of audio DSP **156**, and external audio input received via audio codec **122**). Processor interface **150** provides a data and control interface between main processor **10** and graphics and audio processor **114**.

Memory interface **152** provides a data and control interface between graphics and audio processor **114** and memory **112**. In this example, main processor **110** accesses main memory **112** via processor interface **150** and memory interface **152** that are part of graphics and audio processor **114**. Peripheral controller **162** provides a data and control interface between graphics and audio processor **114** and the various peripherals mentioned above. Audio memory interface **158** provides an interface with audio memory **126**.

Example Graphics Pipeline

FIG. **4** shows a more detailed view of an example 3D graphics processor **154**. 3D graphics processor **154** includes among other things, a command processor **200** and a 3D graphics pipeline **180**. Main processor **110** communicates streams of data (e.g., graphics command streams and display lists) to command processor **200**. Main processor **110** has a two-level cache **115** to minimize memory latency, and also has a write-gathering buffer **111** for uncached data streams targeted for the graphics and audio processor **114**. The write-gathering buffer **111** collects partial cache lines into full cache lines and sends the data out to the graphics and audio processor **114** one cache line at a time for maximum bus usage.

Command processor **200** receives display commands from main processor **110** and parses them—obtaining any additional data necessary to process them from shared memory **112**. The command processor **200** provides a stream of vertex commands to graphics pipeline **180** for 2D and/or 3D processing and rendering. Graphics pipeline **180** generates images based on these commands. The resulting image information may be transferred to main memory **112** for access by display controller/video interface unit **164**—which displays the frame buffer output of pipeline **180** on display **56**.

FIG. **5** is a logical flow diagram of graphics processor **154**. Main processor **110** may store graphics command streams **210**, display lists **212** and vertex arrays **214** in main memory **112**, and pass pointers to command processor **200** via bus interface **150**. The main processor **110** stores graphics commands in one or more graphics first-in-first-out (FIFO) buffers **210** it allocates in main memory **110**. The command processor **200** fetches:

- command streams from main memory **112** via an on-chip FIFO memory buffer **216** that receives and buffers the graphics commands for synchronization/flow control and load balancing
- display lists **212** from main memory **112** via an on-chip call FIFO memory buffer **218**, and
- vertex attributes from the command stream and/or from vertex arrays **214** in main memory **112** via a vertex cache **220**.

Command processor **200** performs command processing operations **200***a* that convert attribute types to floating point format, and pass the resulting complete vertex polygon data to graphics pipeline **180** for rendering/rasterization. A programmable memory arbitration circuitry **130** (see FIG. **4**) arbitrates access to shared main memory **112** between graphics pipeline **180**, command processor **200** and display controller/video interface unit **164**.

FIG. **4** shows that graphics pipeline **180** may include:

- a transform unit **300**,
- a setup/rasterizer **400**,
- a texture unit **500**,
- a texture environment unit **600**, and
- a pixel engine **700**.

Transform unit **300** performs a variety of 2D and 3D transform and other operations **300***a* (see FIG. **5**). Transform unit **300** may include one or more matrix memories **300***b* for storing matrices used in transformation processing **300***a*. Transform unit **300** transforms incoming geometry per vertex from object space to screen space; and transforms incoming texture coordinates and computes projective texture coordinates (**300***c*). Transform unit **300** may also perform polygon clipping/culling **300***d*. Lighting processing **300***e* also performed by transform unit **300***b* provides per vertex lighting computations for up to eight independent lights in one example embodiment. Transform unit **300** can also perform texture coordinate generation (**300***c*) for embossed type bump mapping effects, as well as polygon clipping/culling operations (**300***d*).

Setup/rasterizer **400** includes a setup unit which receives vertex data from transform unit **300** and sends triangle setup information to one or more rasterizer units (**400***b*) performing edge rasterization, texture coordinate rasterization and color rasterization.

Texture unit **500** (which may include an on-chip texture memory (TMEM) **502**) performs various tasks related to texturing including for example:

- retrieving textures **504** from main memory **112**,
- texture processing (**500***a*) including, for example, multi-texture handling, post-cache texture decompression, texture filtering, embossing, shadows and lighting through the use of projective textures, and BLIT with alpha transparency and depth,
- bump map processing for computing texture coordinate displacements for bump mapping, pseudo texture and texture tiling effects (**500***b*), and
- indirect texture processing (**500***c*).

FIGS. **6** through **17** illustrate example texture processing for performing regular (non-indirect) and indirect texture lookup operations. A more detailed description of the example graphics pipeline circuitry and procedures for performing regular and indirect texture look-up operations is disclosed in commonly assigned co-pending patent application Ser. No. 09/722,382, entitled "Method And Apparatus For Direct And Indirect Texture Processing In A Graphics System" and its corresponding provisional application, Ser. No. 60/226,891, filed Aug. 23, 2000, both of which are incorporated herein by reference.

Texture unit **500** outputs filtered texture values to the texture environment unit **600** for texture environment processing (**600***a*). Texture environment unit **600** blends polygon and texture color/alpha/depth, and can also perform texture fog processing (**600***b*) to achieve inverse range based fog effects. Texture environment unit **600** can provide multiple stages to perform a variety of other interesting environment-related functions based for example on color/alpha modulation, embossing, detail texturing, texture swapping, clamping, and depth blending. Texture environment unit **600** can also combine (e.g., subtract) textures in hardware in one pass. For more details concerning the texture environment unit **600**, see commonly assigned application Ser. No. 09/722,367 entitled "Recirculating Shade

Tree Blender for a Graphics System" and its corresponding provisional application, Ser. No. 60/226,888, filed Aug. 23, 2000, both of which are incorporated herein by reference.

Pixel engine **700** performs depth (z) compare (**700***a*) and pixel blending (**700***b*). In this example, pixel engine **700** stores data into an embedded (on-chip) frame buffer memory **702**. Graphics pipeline **180** may include one or more embedded DRAM memories **702** to store frame buffer and/or texture information locally. Z compares **700***a*' can also be performed at an earlier stage in the graphics pipeline **180** depending on the rendering mode currently in effect (e.g., z compares can be performed earlier if alpha blending is not required). The pixel engine **700** includes a copy operation **700***c* that periodically writes on-chip frame buffer **702** to main memory **112** for access by display/video interface unit **164**. This copy operation **700***c* can also be used to copy embedded frame buffer **702** contents to textures in the main memory **112** for dynamic texture synthesis effects. Anti-aliasing and other filtering can be performed during the copy-out operation. The frame buffer output of graphics pipeline **180** (which is ultimately stored in main memory **112**) is read each frame by display/video interface unit **164**. Display controller/video interface **164** provides digital RGB pixel values for display on display **102**.

Example Environment-Mapped Bump Mapping Procedures

FIG. **18** shows an example general environment mapped bump-mapping procedure. At a high level of abstraction, the preferred implementation of the environment-mapped bump mapping procedure of the present invention generates texture coordinates from surface parameters such as the surface Normal and binormals (e.g., Normal and/or Tangent and Binormal vectors) (Block **1400**A). These surface parameters can be provided by transform unit **300** based on lighting calculations or by main processor **110**, for example.

Texture coordinates are generated based on the surface parameters, and an indirect bump map texture is used to offset (perturb) the resulting texture coordinates (block **1400**B). The perturbed texture coordinates are then used to look up an environment texture map (block **1400**C). The environment texture map may contain complicated lighting effects, or it may be, for example, a spherical reflection map of the environment or any other desired environment map or maps. The resulting image may then be further processed and displayed (block **1400**D).

Environment-mapped bump mapping (EMBM) in accordance with the present invention is accomplished in a preferred implementation through the use of indirect texture processing as described in the above reference copending patent application. This allows multiple texture lookups in a single rendering pass using low cost hardware.

There are at least two ways to model surface perturbations in a bump map in the example described above. One way is to use a three-dimensional model (e.g., a sphere, toroid, etc.) that matches (or closely approximates) the surface geometry of the 3D object being mapped. In this approach, a bump map of the three-dimensional model of the object describes height differences in three dimensions (x, y, z) relative to the surface being mapped. These differences in three dimensions (which may be represented by partial derivatives in each of the three coordinates corresponding to s, t, u texture coordinates) can be converted into eye space, translated (e.g., in the case of object rotation) and then used to perturb texture coordinates generated in the conventional fashion from a surface Normal to provide new texture coordinates for Gouraud-shaded bump-mapping.

Another way model surface perturbations in a bump map is to use a two-dimensional (i.e., flat surface) model. In this approach, the model need not correspond precisely to the surface geometry of the 3D object being mapped. A bump map of the two-dimensional model can describe height differences in two dimensions (x, y) relative to the surface being mapped. These height differences in two dimensions, which can be represented by partial derivatives in x and y, may be stored in a texture memory as partial derivatives (dSdT) and accessed using s and t texture coordinates. These partial derivatives may then be used to texture mapping coordinates generated from the surface binormals (i.e., the Tangent and Binormal vectors) and those values and the texture coordinates generated from the surface Normal vector are combined to produce perturbed Normal vector texture coordinates for Gouraud-shaded bump mapping.

In the first example implementation, Normal vector perturbations are modeled in 3D (dXdYdZ) with respect to a particular geometric shape (e.g., sphere, toroid, etc.) and, during application program run-time, are matched to corresponding similar geometric surfaces and transformed into eye-space. This method has the advantage that it renders bump-mapped images very quickly because it requires few coordinate processing stages to compute. In the second example implementation, perturbations of the Normal are modeled with respect to a flat surface (dSdT) and during run time may be mapped onto arbitrary object surfaces. This second method of implementation is more versatile but requires more coordinate processing stages and additional binormal surface vector information to calculate the perturbed Normal texture coordinates (see, for example, the above referenced copending application regarding coordinate processing and TEV stages for performing indirect texture referencing operations).

Environment-Mapped Bump-Mappinq Based On Perturbations Modeled in 3D (dXdYdZ):

FIG. **19** shows a logical block diagram illustrating an example process of environment-mapped bump mapping using Normal vector perturbations modeled in 3D (dXdYdZ). In one preferred example embodiment, environment mapping can be performed using a spherical reflection map. The environment-mapped bump-mapped dXdYdZ texture lookup that is performed is analogous to a perturbed Normal lookup into a spherical reflection map. (As an "indirect" operation, this is analogous to a texture warp operation using signed offsets.) Normal vector perturbations for environment mapped bump mapping are computed on a per-vertex basis. In this example, the indirect bump map **1401** contains precomputed Normal vector offsets in 3D model space (i.e., object space). A surface Normal vector at a vertex is initially rotated (i.e., converted) to eye-space. Texture coordinate data generated from the x and y eye-space components of the converted Normal vector are then scaled (multiplied) by offsets (i.e., partial derivative data dF/dx, dF/dy) obtained from the dXdYdZ bump map texture to generate a new set of Normal texture coordinates (i.e., perturbated texture coordinates n'x and n'y) for performing a texture lookup to an environment or light map.

As indicated in the logical diagram of FIG. **19**, an indirect texture look-up operation is performed using indirect coordinate s_0, t_0 and direct texture look-up is performed using coordinates s_1, t_1. As a result of the indirect texture look-up, offset data s, t and u (**1402**) is retrieved from the dXdYdZ bump map **1401** via, for example, the multi-bit R, G and B color channel lines over col bus **519** (FIG. **15**). In this example, only the s and t offset data, corresponding to the partial derivatives dF/dx and dF/dy of the bump function F, is used in computing the perturbed normal component texture coordinates. During the next coordinate processing

cycle/stage, texture coordinate input values, s_1, t_1 (**1403**), generated from surface Normal vector components (i.e., nx and ny), are scaled (multiplied) by the retrieved s and t offset data **1402**. For this example, the nx and ny (s_1 and t_1) input data is provided in the same manner as regular (non-indirect) coordinates to the bump processing unit **512** and is loaded into matrix M (**1404**) for multiplication by partial derivative offset data V (**1405**) from the indirect texture lookup. The resulting scaled coordinate products, n'x (**1406**) and n'y (**1407**), provide the effect of a perturbed normal and are used to perform an environment map lookup **1408**. For the above example implementation, the 3×2 matrix M in bump processing unit **512** (FIG. **15**) may be loaded with the appropriate matrix elements for effecting a transform from model space to eye-space and, if desired, rotation of the bump-mapped object about any of plural axes. The application program should set up a Normal-based-texture coordinate generation (i.e., an API texgen function) for the regular (non-indirect) coordinates s_1 and t_1.

Environment-Mapped Bump-Mapping Based on Perturbations Modeled with Respect to a Flat Surface (dSdT):

FIG. **20** shows an example simplified flat surface environment mapped bump mapping operation. In this example, texture coordinates s and t, corresponding to a position on the surface being mapped, are used to perform an indirect texture lookup **1410**. The indirect texture specifies db/ds, db/dt partial derivatives representing height differences defined by a bump map function. These partial derivatives may be outputted by texture unit **500** via, for example, multibit red (R) and green (G) color channel lines over col bus **514** (FIG. **15**). FIG. **21** shows an example bump map texture providing such partial derivatives, and FIG. **22** illustrates a corresponding bump map function.

The partial derivatives provided by the indirect texture lookup are supplied to bump unit **500***b*. FIG. **23** shows a simplified example of what bump unit **500***b* door effectively accomplishes using these partial derivatives. Effectively, Bump unit **500***b* multiplies the object surface binormals (actually texture coordinates generated from the binormals) by the partial derivatives and then sums the results, i.e.:

(b0* db/ds)+(b1 * db/dt), where b0 and b1 are the object surface binormals (i.e., Tangent and Binormal vectors). Bump unit **500***b* then uses this result to effectively perturb the surface Normal (e.g., by summing). The resulting perturbed normal (actually perturbed texture coordinates) is then used for environment mapping **1434**.

FIG. **24** is a more detailed logical block diagram illustrating an example method of environment-mapped bump mapping using Normal vector perturbations modeled with respect to a flat surface (dSdT). In one example embodiment, the environment mapping is performed using a spherical reflection map. The environment mapped bump mapping dSdT texture lookup that is performed is analogous to a perturbed Normal lookup into the spherical reflection map. The Normal vector coordinate perturbations are computed on a per-vertex basis. These perturbations are based on the Tangent, Binormal and Normal vector generated texture coordinates (S_2, t_2), (S_3, t_3), (S_4, t_4) and partial derivative information obtained from an indirect texture bump map. For this implementation, the indirect bump map **1410** contains precomputed deltas for S and T (i.e., partial derivative information offsets), modeled with respect to a flat surface (dSdT). The surface Normal, Binormal and Tangent vectors per-vertex are rotated to eye-space and texture coordinates (s_2, t_2) **1412**, (S_3, t_3) **1414**, and (S_4, t_4) **1416** are generated from the x and y eye-space components of each vector ((T_x, T_y), (B_x, B_y), (N_x, N_y. The Tangent and Binormal generated texture coordinates (s_2, t_2) **1412** and (S_2, t_3) **1414** are scaled (multiplied) (**1413**, **1415**) by pre-computed partial derivative offsets (F_u, F_v) (**1417**) retrieved during an indirect texture lookup to indirect texture bump map **1410**. Resulting x-component coordinate products F_u B_x, (**1418**) and F_v,T_x) (**1420**) are combined (added) (**1430**) with x-component coordinate, N_x (**1422**), to provide a perturbed Normal x-component texture coordinate n'x. Likewise, the resulting y-component coordinate products F_v T_y (**1424**) and $F_u B_y$ (**1426**) are combined (**1432**) with the y-component coordinate, N_y (**1428**) to provide a perturbed Normal y-component texture coordinate n'y. The two perturbed Normal texture offset coordinates n'x and n'y are then used in performing a regular (non-indirect) texture lookup into one or more texture environment or light maps (**1434**).

FIG. **25** shows example steps for performing environment mapped bump mapping using a flat surface model. In this example, a first stage looks up the bump map (block **1502**). Second and third stages process the binormals (blocks **1504**, **1506**). A fourth stage processes the Normal and looks up light map **0** (for example, specular lighting) (block **1508**). A fifth stage looks up light map **1** (for example, diffuse lighting) (block **1510**). It is not necessary to have two light maps—different numbers of light maps could be used if desired.

FIG. **26** is a flowchart showing this overall procedure in one example implementation. Block **1440** creates or provides the bump map and environment light maps, and generates the partial derivatives (which may be dynamically calculated by the application running on main processor **110**) for storing into an indirect texture.

Block **1441** supplies the Normal, Binormal and Tangent vectors per vertex (this can be performed by transform unit **300**). In one example embodiment, these values can be separately indexed per vertex to compress the amount of data needed for bump mapping. All of the vectors can share one base pointer.

Block **1441** also can rotate the vectors to eye space, and provides x and y coordinates. This can be done by setting up a Normal-based texture generation for the regular (non-indirect) texture coordinates (S_2, t_2) (S_3, t_3), (S_4, t_4) in FIG. **24**. The texture coordinates are iterated across the polygon and supplied to the indirect texture mapping hardware for each pixel (block **1441**).

Block **1442** uses the texture coordinates s_1, t_2 to look up the partial derivative (height offset) data from the partial derivative bump map. These retrieved values are multiplied by the Tangent coordinates s_2, t_2 and the results are retained by the bump unit **500***b*. Block **1443** multiplies the Binormal coordinates S_3, t_3 by these same partial derivatives and added to the previous products. Block **1444** then adds the sum-of-products to the Normal coordinates S_4, t_4 to generated perturbed Normal coordinates for environment mapping. Block **1445** can reuse these values to look up any number of additional light or environment maps. The environment mapping are then blended with the surface illumination in a conventional manner (block **1446**) and the results are displayed.

FIG. **27** shows a more detailed flowchart of how the particular hardware implementation is stepped through the FIG. **26** process. The following table illustrates one example of the order of operations performed during successive processing stages for EMBM based on perturbations modeled with respect to a flat surface (dSdT) and performing multiple texture mappings:

	bump unit processing:	TEV Stages:
1.	Tangent vector ($\overline{T}$): (binormal 0) · Fv	(Disable texture access)
2.	Binormal vector ($\overline{B}$): (binormal 1) · Fu + previous result (recirculated)	(Disable texture access)
3.	Normal + previous result (recirculated): ($\overline{N}$ + Fv $\overline{T}$ + Fu $\overline{B}$)	lookup to lightmap 0 (e.g., specular)
4.	Previous result (reuse)	lookup to lightmap 1 (e.g., diffuse)
5.	S, T (reuse previous result)	surface texture (lookup to environment texture)

In the above example, TEV stages **3** through **5** would perform blending as follows:

lightmap0+lightmap1+surface texture

(specular) (diffuse)

Theory of Operation

As explained above, bump mapping allows one to add realism to an image without using a lot of geometry. Basically, bump mapping modifies the shading of a polygon by effectively perturbing the surface normal on a per-pixel basis. The shading makes the surface appear bumpy, even though the underlying geometry is relatively flat. The following discussion briefly describes the basic mathematics of bump mapping:

Bump Mapping Mathematics:

In general, the "bumpiness" of a surface can be described by a 2D height field, also called a "bump map". The bump map is generally defined by a scalar difference F(u, v) between a flat surface P(u, v) and a desired bumpy surface P'(u, v) along the normal N at each point (u, v). Thus, the bump surface may be represented as:

$$P'(u, v)=P(u, v)+F(u,v)\bar{N}$$

Normally, P is modeled using polygons, and F is modeled as a 2D image using a paint program or other tool. The normal Vector N' at (u, v) can be calculated by the cross product of the partial derivatives of P' in u and v. For this discussion, the partial derivative terms are defined by the following simplified notation:

$$P_u = \frac{\partial P(u, v)}{\partial u},\ P_v = \frac{\partial P(u, v)}{\partial_v},\ P'_u = \frac{\partial P'(u, v)}{\partial u},\ P'_v = \frac{\partial P'(u, v)}{\partial v},$$

$$F_u = \frac{\partial F(u, v)}{\partial u},\ F_v = \frac{\partial F(u, v)}{\partial v}$$

Accordingly, the partial derivatives of P'(u, v) can be computed using the chain rule from $(P'_u \times P'_v)=\overline{N}'$ and P'(u, v)=P(u, v)+F(u, v)$\overline{N}$ as follows:

$$P'_u = P_u + F_u\overline{N} + F\frac{\partial \overline{N}}{\partial u}$$

$$P'_v = P_v + F_v\overline{N} + F\frac{\partial \overline{N}}{\partial v}$$

If F is assumed to be sufficiently small, the last term in the above two equations can be approximated by zero and, thus, the partial derivatives may be represented as:

$$P'_u=P_u+F_u\overline{N} \text{ and}$$

$$P'_v=P_v+F_v\overline{N}$$

Taking the cross product gives the following expression for N':

$$\overline{N}'=P_u\times P_v+F_u(\overline{N}\times P_u)+F_v(\overline{N}\times P_v)+F_uF_v(\overline{N}\times\bar{N})$$

Since by definition,

$$\overline{N}=P_u\times P_v \text{ and } \overline{N}\times\overline{N}=0$$

the equation for N' can be simplified to:

$$\overline{N}'=\overline{N}+F_u(\overline{N}\times P_u)+F_v(\overline{N}\times P_v)$$

Using the notation of $\overline{T}$ for the tangent vector and $\overline{B}$ for the Binormal vector:

$$\overline{B}=\bar{N}\times P_u \text{ and } \overline{T}=\bar{N}\times P_v,$$

a perturbed Normal may be computed according to the following equation:

$$\overline{N}'=N+F_u\overline{B}+F_v\overline{T}$$

The values Fu and Fv may be computed offline, for example, through "forward differencing" of the 2D bump map. Pu and Pv may be computed either directly from the surface definition, or from forward differencing applied to the surface parameterization. Forward differencing is a known conventional method which may be used to compute (approximate) the first derivative of a height field (bump map) in a given direction. For example, assuming that a one-dimensional bump map is represented as the height function F(s), the forward difference would be represented by the following:

$$F_1=F(s)-F(s+\Delta s)$$

If, for example, the delta is one texel in s, and the width of the bump map is w, then the forward difference is (assuming s is normalized to map coordinates):

$$F_1 = F(s) - F\left(s + \frac{1}{w}\right)$$

If F were continuous, the above equations would approximate the first derivative of F. For a 2D bump map, a forward differencing operation may be performed by the following pseudo-code example:

```
For (t=0; t<h; t++) {
  For(s=0;s<w;s++){
    tex00=bump__imag(s,t);
    tex01=bump__imag(s+1, t);
    tex10=bump__imag(s, t+1);
    Fs=tex00-tex01;
    Ft=tex00-tex10;
  }
}
```

Example API Environment-mapped Bump-mapping Function Commands:

As shown in FIGS. **12–13**, one or more graphics API functions are used to set up and initiate deviational indirect texture look-up operations. Example API functions for setting up indirect texture operations and parameters for performing environment-mapped bump-mapping may be defined as follows:

GXSetTevIndBumpXYZ

This function sets up an environment-mapped bump-mapped dXdYdZ texture indirect lookup. The indirect map specifies offsets in object (X, Y, Z) space. This kind of

lookup requires only one TEV stages to compute, the indirect matrix should be loaded with a transformation for normals from object space to eye space. The surface geometry need only provide regular normals at each vertex.

Example Arguments:

Tev_stage The TEV stage that is being affected

Ind_stage The indirect stage results to use with this TEV stage.

Matrix-sel Indicates which indirect matrix and scale value to use to multiply the retrieved offsets.

Example Usage:

void GXSetTevIndBumpXYZ(GXTevStageID tev_stage,
GXIndTexStageID ind_stage,
GXIndTexMtxId matrix_sel);

GXSetTevIndBumpST

This function sets up an environment-mapped bump-mapped dSdT texture indirect lookup. The indirect map specifies offsets in (S, T) space. This lookup function requires 3 TEV stages to compute the offset texture coordinates. The resulting texture coordinates are available two stages after the one specified in the function call. The first two TEV stages should disable texture lookup. The third stage is where the lookup is actually performed. One may use an API GXSetTevIndRepeat function (described below) in subsequent TEV stages to reuse the computed texture coordinates for additional lookups. The surface geometry should provide Normal/Binormal/Tangent vectors at each vertex. This function makes use of the dynamic form of matrices (FIG. **16**B) available in the bump processing unit (FIG. **15**) for transforming the S and T offsets to be relative to the incoming regular S and T (which come from the object normals). A scale value provided by the bump unit should contain the size of the reflection map divided by 2 (and thus the reflection map should be a square power of 2). In this operation the static offset matrix (FIG. **16**A) is not Fused for the texture coordinate computation. Instead, one of two dynamic matrices (i.e., matrix A and matrix B of FIG. **16**B) is used along with a scaling value. The application program should disable the first two TEV blending stages from texture look-ups.

Example Arguments:

Tev_stage The TEV stage that is being affected.

Ind_stage The indirect stage results to use with this TEV stage.

Matrix_sel Indicates which indirect matrix scale value to use to multiply the retrieved offsets.

Example Usage:

void GXSetTevIndBumpSt. (GXTevStageID tev_stage,
GXIndTexStageID ind_stage,
GXIndTexMtxId matrix_sel);

Having used 3 TEV stages to compute a texture coordinate for an EMBM dSdT lookup, one can use the result to do more than one lookup to different texture maps in memory. For example, one may perform additional lookups for specular and diffuse lightmaps. In order to perform successive texture map lookups without taking 3 stages to compute each one, one may use the texture coordinate feedback feature of the present invention.

GXSetTevIndRepeat

This function is used when one wishes to use the same texture coordinates for one TEV stage as were computed in the previous stage. This is useful when the texture coordinates require more than one stage to compute, as is the case for GXSetTevIndBumpST operations.

Example Arguments:

tev_stage The TEV stage that is being changed.

Example Usage:

void GXSetTevIndRepeat (GXTevStageID tev_stage);

Other Example Compatible Implementations

Certain of the above-described system components **50** could be implemented as other than the home video game console configuration described above. For example, one could run graphics application or other software written for system **50** on a platform with a different configuration that emulates system **50** or is otherwise compatible with it. If the other platform can successfully emulate, simulate and/or provide some or all of the hardware and software resources of system **50**, then the other platform will be able to successfully execute the software.

As one example, an emulator may provide a hardware and/or software configuration (platform) that is different from the hardware and/or software rn configuration (platform) of system **50**. The emulator system might include software and/or hardware components that emulate or simulate some or all of hardware and/or software components of the system for which the application software was written. For example, the emulator system could comprise a general purpose digital computer such as a personal computer, which executes a software emulator program that simulates the hardware and/or firmware of system **50**.

Some general purpose digital computers (e.g., IBM or MacIntosh personal computers and compatibles) are now equipped with 3D graphics cards that provide 3D graphics pipelines compliant with DirectX or other standard 3D graphics command APIs. They may also be equipped with stereophonic sound cards that provide high quality stereophonic sound based on a standard set of sound commands. Such multimedia-hardware-equipped personal computers running emulator software may have sufficient performance to approximate the graphics and sound performance of system **50**. Emulator software controls the hardware resources on the personal computer platform to simulate the processing, 3D graphics, sound, peripheral and other capabilities of the home video game console platform for which the game programmer wrote the game software.

FIG. **28** illustrates an example overall emulation process using a host platform **1201**, an emulator component **1303**, and a game software executable binary image provided on a storage medium **62**. Host **1201** may be a general or special purpose digital computing device such as, for example, a personal computer, a video game console, or any other platform with sufficient computing power. Emulator **1303** may be software and/or hardware that runs on host platform **1201**, and provides a real-time conversion of commands, data and other information from storage medium **62** into a form that can be processed by host **1201**. For example, emulator **1303** fetches "source" binary-image program instructions intended for execution by system **50** from storage medium **62** and converts these program instructions to a target format that can be executed or otherwise processed by host **1201**.

As one example, in the case where the software is written for execution on a platform using an IBM PowerPC or other specific processor and the host **1201** is a personal computer using a different (e.g., Intel) processor, emulator **1303** fetches one or a sequence of binary-image program instructions from storage medium **62** and converts these program instructions to one or more equivalent Intel binary-image program instructions. The emulator **1303** also fetches and/or generates graphics commands and audio commands intended for processing by the graphics and audio processor

114, and converts these commands into a format or formats that can be processed by hardware and/or software graphics and audio processing resources available on host **1201**. As one example, emulator **1303** may convert these commands into commands that can be processed by specific graphics and/or or sound hardware of the host **1201** (e.g., using standard DirectX, OpenGL and/or sound APIs).

An emulator **1303** used to provide some or all of the features of the video game system described above may also be provided with a graphic user interface (GUI) that simplifies or automates the selection of various options and screen modes for games run using the emulator. In one example, such an emulator **1303** may further include enhanced functionality as compared with the host platform for which the software was originally intended.

In the case where particular graphics support hardware within an emulator does not include the example environment-mapped bump-mapping and indirect texture referencing features and functions illustrated by FIGS. **6** through **27**, the emulator designer has a choice of either:

translating the environment-mapped bump-mapping and indirect-texture referencing commands into other graphics API commands the graphics support hardware understands, or

implementing environment-mapped bump-mapping and indirect-texture referencing in software with a potential corresponding decrease in performance depending upon the speed of the processor, or

"stubbing" (i.e., ignoring) the environment-mapped bump-mapping and indirect-texture referencing commands to provide a rendered image that does not include effects utilizing environment-mapped bump-mapping and indirect-texture referencing.

While the logical diagrams of FIGS. **6** through **24** can be implemented entirely in software, entirely in hardware or by a combination of hardware and software, the preferred embodiment performs most of these calculations in hardware (e.g., using bump unit **500***b*) to obtain increased speed performance and other advantages. Nevertheless, in other implementations (e.g., where a very fast processor is available), some or all of the processing described herein may be implemented in software to provide similar or identical imaging results.

FIG. **29** illustrates an emulation host system **1201** suitable for use with emulator **1303**. System **1201** includes a processing unit **1203** and a system memory **1205**. A system bus **1207** couples various system components including system memory **1205** to processing unit **1203**. System bus **1207** may be any of several types of bus structures including a memory bus or memory controller, a peripheral bus, and a local bus using any of a variety of bus architectures. System memory **1197** includes read only memory (ROM) **1252** and random access memory (RAM) **1254**. A basic input/output system (BIOS) **1256**, containing the basic routines that help to transfer information between elements within personal computer system **1201**, such as during start-up, is stored in the ROM **1252**. System **1201** further includes various drives and associated computer-readable media. A hard disk drive **1209** reads from and writes to a (typically fixed) magnetic hard disk **1211**. An additional (possible optional) magnetic disk drive **1213** reads from and writes to a removable "floppy" or other magnetic disk **1215**. An optical disk drive **1217** reads from and, in some configurations, writes to a removable optical disk **1219** such as a CD ROM or other optical media. Hard disk drive **1209** and optical disk drive **1217** are connected to system bus **1207** by a hard disk drive interface **1221** and an optical drive interface **1225**, respectively. The drives and their associated computer-readable media provide nonvolatile storage of computer-readable instructions, data structures, program modules, game programs and other data for personal computer system **1201**. In other configurations, other types of computer-readable media that can store data that is accessible by a computer (e.g., magnetic cassettes, flash memory cards, digital video disks, Bernoulli cartridges, random access memories (RAMs), read only memories (ROMs) and the like) may also be used.

A number of program modules including emulator **1303** may be stored on the hard disk **1211**, removable magnetic disk **1215**, optical disk **1219** and/or the ROM **1252** and/or the RAM **1254** of system memory **1205**. Such program modules may include an operating system providing graphics and sound APIs, one or more application programs, other program modules, program data and game data. A user may enter commands and information into personal computer system **1201** through input devices such as a keyboard **1227**, pointing device **1229**, microphones, joysticks, game controllers, satellite dishes, scanners, or the like. These and other input devices can be connected to processing unit **1203** through a serial port interface **1231** that is coupled to system bus **1207**, but may be connected by other interfaces, such as a parallel port, game port Fire wire bus or a universal serial bus (USB). A monitor **1233** or other type of display device is also connected to system bus **1207** via an interface, such as a video adapter **1235**.

System **1201** may also include a modem **1154** or other network interface means for establishing communications over a network **1152** such as the Internet. Modem **1154**, which may be internal or external, is connected to system bus **123** via serial port interface **1231**. A network interface **1156** may also be provided for allowing system **1201** to communicate with a remote computing device **1150** (e.g., another system **1201**) via a local area network **1158** (or such communication may be via wide area network **1152** or other communications path such as dial-up or other communications means). System **1201** will typically include other peripheral output devices, such as printers and other standard peripheral devices.

In one example, video adapter **1235** may include a 3D graphics pipeline chip set providing fast 3D graphics rendering in response to 3D graphics commands issued based on a standard 3D graphics application programmer interface such as Microsoft's DirectX 7.0 or other version. A set of stereo loudspeakers **1237** is also connected to system bus **1207** via a sound generating interface such as a conventional "sound card" providing hardware and embedded software support for generating high quality stereophonic sound based on sound commands provided by bus **1207**. These hardware capabilities allow system **1201** to provide sufficient graphics and sound speed performance to play software stored in storage medium **62**.

All documents referenced above are hereby incorporated by reference.

While the invention has been described in connection with what is presently considered to be the most practical and preferred embodiment, it is to be understood that the invention is not to be limited to the disclosed embodiment, but on the contrary, is intended to cover various modifications and equivalent arrangements included within the scope of the appended claims.

What is claimed is:

1. In a graphics system that renders and displays images at least in part in response to polygon vertex data and texture data stored in memory, said system capable of direct and

indirect mode texture coordinate processing, a method of environment-mapped bump mapping based on a bump map function modeled with respect to a flat surface, comprising the steps of:

(a) supplying Normal, Binormal and Tangent vector information per vertex as vertex attribute data associated with an object surface and generating texture coordinates based on said vector information;

(b) using a set of indirect mode texture coordinates to reference bump map data stored as texture data in memory;

(c) combining bump map data referenced in step (b) with texture coordinates generated from said Normal, Binormal and Tangent surface vector information to compute vector-perturbed texture coordinates based at least in part on the bump map data;

(d) saving a set of computed perturbed texture coordinates for reuse in performing multiple lookups to a texel in a plurality of different color and/or image data maps stored as texture data in memory;

(e) performing a lookup to a texel in a light map stored as texture data in memory to retrieve lighting data for shading a pixel, wherein the lookup is based on computed perturbed texture coordinates saved in step (d);

(f) performing a lookup to a texel in an environment map stored as texture data in memory to retrieve color data for shading said pixel, wherein the lookup is based on perturbed texture coordinates saved in step (d) and used in step (e); and

shading said pixel based on lighting data and color data acquired from at least said light map and said environment map;

wherein steps (a) through (f) are performed during a single rendering pass.

2. A method of environment-mapped bump mapping as set forth in claim **1** wherein a graphics vertex command function specifies the Normal, Tangent and Binormal attributes per vertex, or specifies a separate index to each such attributes stored in memory.

3. A method of environment-mapped bump mapping as set forth in claim **1** wherein the bump map data stored as a texture comprises bump function partial derivative data (dSdT) or bump function forward differencing data representing height differences defined by a surface bump map function modeled in two dimensions with respect to a flat surface.

4. A method of environment-mapped bump mapping as set forth in claim **1** wherein said saved computed perturbed texture coordinates are reused during the same rendering pass to retrieve texels from two or more of different light maps.

5. A method of environment-mapped bump mapping as set forth in claim **1** wherein said computed perturbed texture coordinates are saved in a storage buffer associated with texture coordinate processing hardware operating within the graphics system.

6. A method of environment-mapped bump mapping as set forth in claim **1** wherein the bump map data stored as a texture comprises precomputed forward difference values, F_u and F_v, of a surface bump function F(u, v) modeled in two dimensions with respect to a flat surface.

* * * * *